

SECR2033
Computer Organization and Architecture

Lecture slides prepared by “Computer Organization and Architecture”, 9/e, by William Stallings, 2013.

Module 6
Memory

Objectives:

- To understand the concepts of hierarchical memory organization.
- To understand how each level of memory contributes to system performance, and how the performance is measured.
- To understand the concepts behind cache memory and the basic concept of virtual memory.

Module 6
Memory

6.1 Introduction
6.2 Main Memory
6.3 Cache Memory
6.4 Virtual Memory
6.5 Summary

Module 6
Memory

6.1 Introduction
6.2 Main Memory
6.3 Cache Memory
6.4 Virtual Memory
6.6 Summary

- Overview
- Type of Memory
- The Memory Hierarchy

6 Overview

- Most computers are built using the *Von Neumann* model, which is centered on memory.
- The *programs* that perform the processing are stored in memory.
- The memory unit that communicates directly with the CPU is called *main memory**.
- Devices that provide backup storage are called *auxiliary memory*.*

Only programs and data currently needed by the processor reside in main memory.

Linda Null and Julia Lobur (2003). *The Essentials of Computer Organization and Architecture.* United States: Jones and Bartlett Publishers. p.233.
* Mano, Morris M. (1993). *Computer System Architecture* (3rd Edition). p.445.

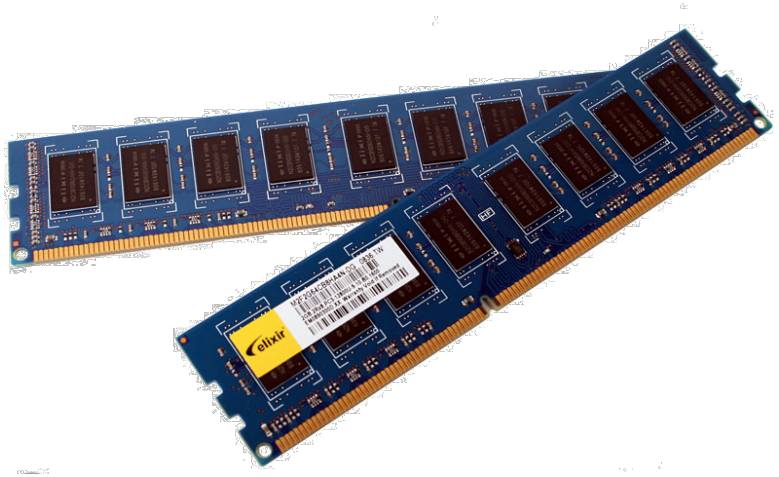
- Internal memory is often referred with main memory, but there are other forms of internal memory:
  - The processor requires its own local memory → registers.
  - The control unit portion of the processor may also require its own internal memory → control memory.
  - Cache is another form of internal memory.

Type of Memory

- Even though a large number of memory technologies exist, there are only two basic types of memory:

RAM (Random Access Memory)

ROM (Read-Only Memory)



(a) RAM *(Random Access Memory)*

- RAM is also the “main memory” or primary memory.
- Used to store programs and data that the computer needs when executing programs.



However, RAM is volatile, and loses this information once the power is turned off.

- Two general types of chips used to build the bulk of RAM memory in today’s computers:
  - DRAM (*Dynamic Random Access Memory*).
  - SRAM (*Static Random Access Memory*).

Linda Null and Julia Lobur (2003). *The Essentials of Computer Organization and Architecture.* United States: Jones and Bartlett Publishers. p.234
* William Stallings (2016). *Computer Organization and Architecture: Designing for Performance* (10th Edition). United States: Pearson Education Limited, p.124

- **D**ynamic RAM (DRAM)
  - Commonly used as main memory
  - Use capacitor to store data, 1- charged, 0 – discharged
  - Capacitor will lose its charge with time ➔ need to recharge (refresh)
- **S**tatic RAM (SRAM)
  - Used for cache memory.
  - Use flip-flop to store data – no need refresh
  - Compared to DRAM – SRAM is faster but more expensive, more complex and low capacity

(b) ROM *(Read-Only Memory)*

- Stores critical information necessary to operate the system, such as the program necessary to boot the computer.



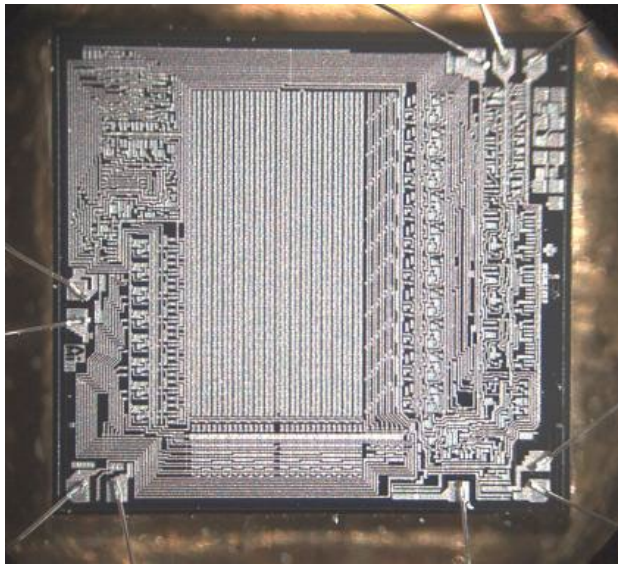
- ROM is not volatile and always retains its data.
- Maintain information when the power is shut off.

- Some different types of ROM:
  - PROM *(Programmable ROM)*
  - EPROM (*Erasable PROM*)
  - EEPROM (*Electrically Erasable PROM*)
  - Flash memory → essentially EEPROM with the added benefit .

- **P**rogrammable **ROM** (PROM)
  - Programmed after manufactured
  - Once they are programmed, cannot be changed (**O**ne **T**ime **P**rogrammable)
- **E**rasable **P**rogrammable **ROM** (EPROM)
  - can be erased by exposing to Ultraviolet (UV) radiation for a few minutes
  - can be reprogrammed
- **E**lectrically **E**rasable and **P**rogrammable **ROM** (EEPROM)
  - Erase electrically, but not using UV
  - $\therefore$ no need to take out the IC to erase

ROM Usage

- Microprogramming
- Library subroutines
- Systems programs (BIOS)
- Function tables



HP-35 ROMs.
The array contains 2560 bits.

6 The Memory Hierarchy

- In general, faster memory is more expensive than slower memory.
- To provide the best performance at the lowest cost, memory is organized in a hierarchical fashion.

- Small, fast storage elements are kept in the CPU (i.e., registers and caches).
- Larger, slower main memory (RAM) is accessed through the data bus.
- Largest, (almost) permanent storage in the form of disk and backup drives is further from the CPU.

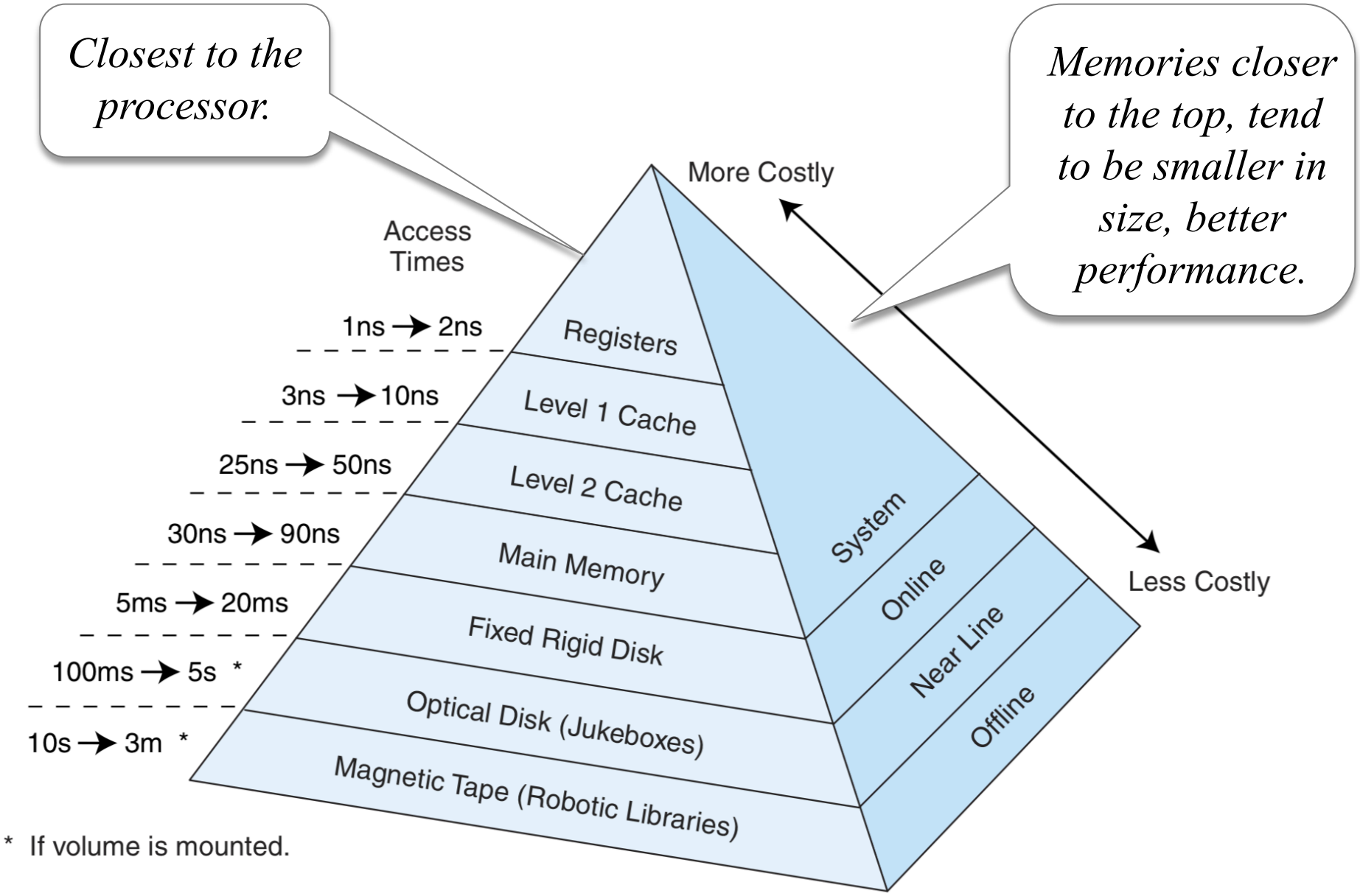
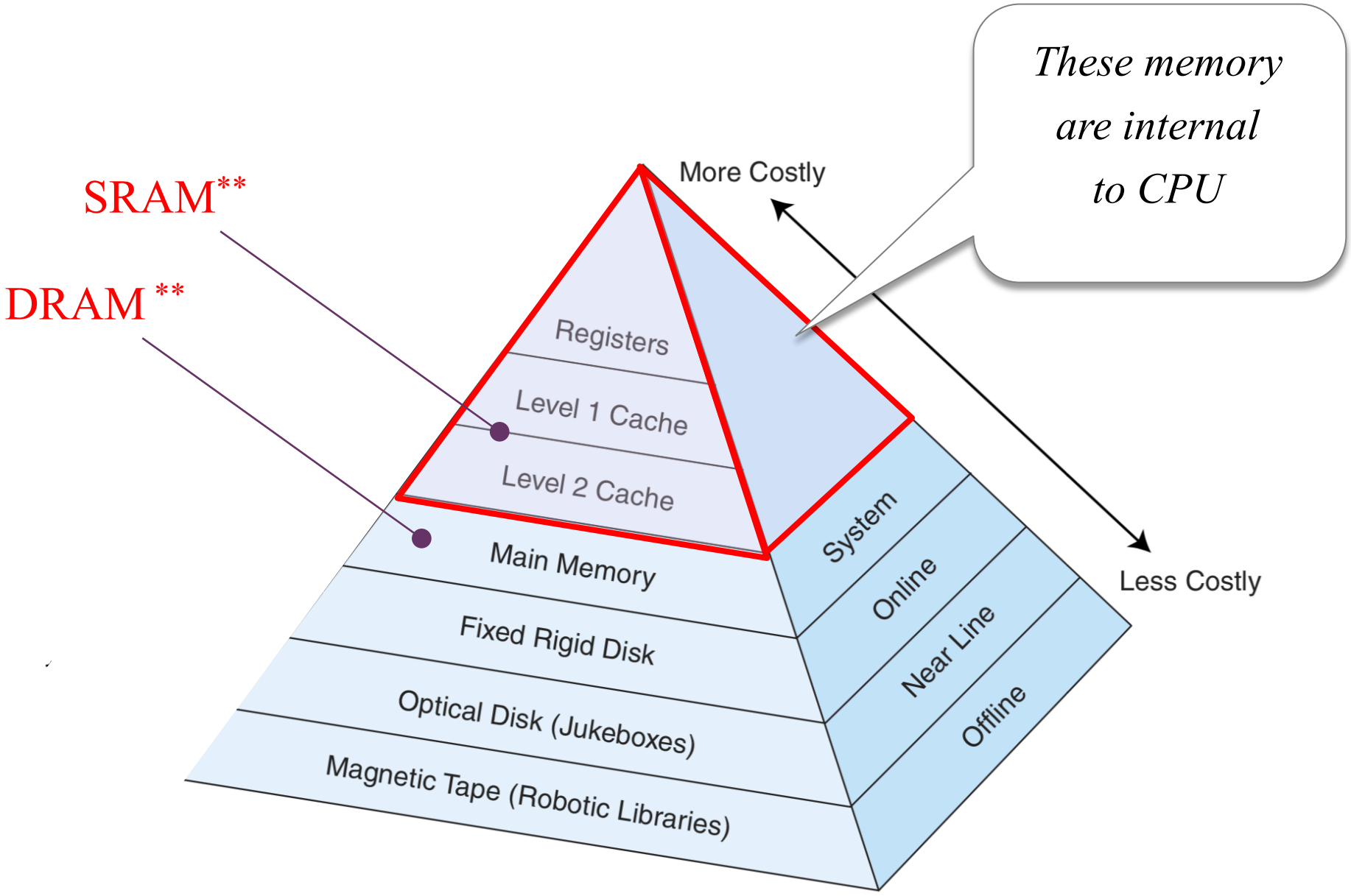


Figure: The memory hierarchy.



Linda Null and Julia Lobur (2003). *The Essentials of Computer Organization and Architecture.* United States: Jones and Bartlett Publishers. p.236
* William Stallings (2016). *Computer Organization and Architecture: Designing for Performance* (10th Edition). United States: Pearson Education Limited, p.127
** Patterson, D.A. and Hennessy, J.L. (2014). *Computer Organization and Design: The Hardware/Software Interface (5th* Edition). United States: Elsevier, p.378

- To access a particular piece of data, the CPU first sends a request to its nearest memory, which is cache.
  - If the data is not in cache, then main memory is queried.
  - If the data is not in main memory, then the request goes to the disk (HDD / SSD).

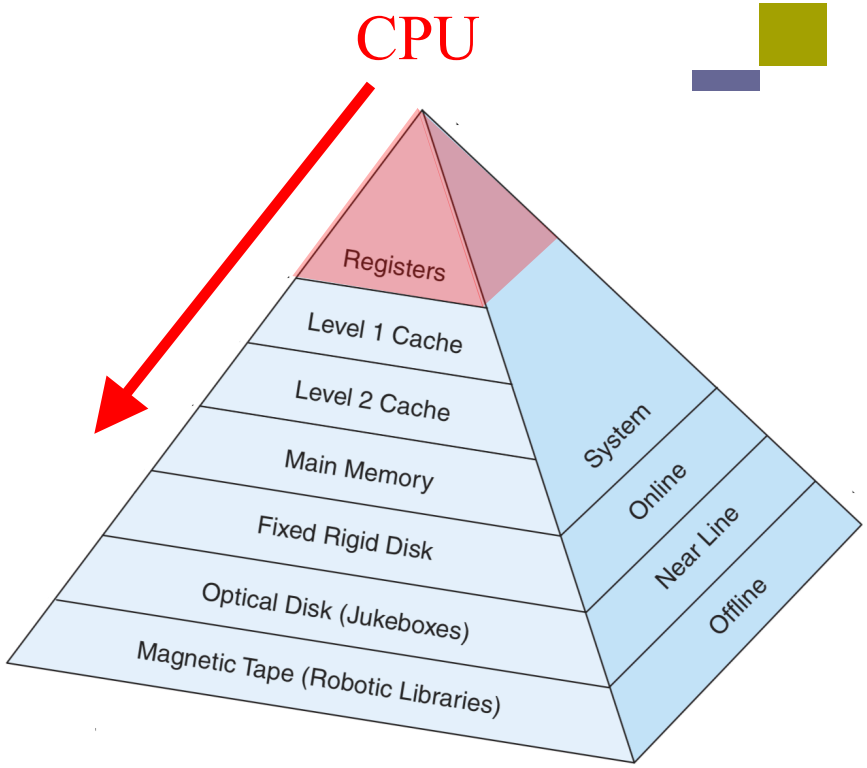


Table: The following terminology is used when referring to the memory hierarchy.

Terminology	Definition
Hit	The data is found at a given memory level.
Miss	The data is not found at a given memory level.
Hit Rate	The percentage (%) of memory accesses found in a given level of memory.
Miss Rate	The percentage (%) of memory accesses not found in a given level of memory → (1 – *hit rate*)
Hit Time	The time required to access data at a given memory level.
Miss Penalty	The time required to process a *miss*, including the time that it takes to replace a block of memory plus the time it takes to deliver the data to the processor.

Module 6
Memory

6.1 Introduction

6.2 Main Memory

6.3 Cache Memory

6.4 Virtual Memory

6.6 Summary

- Overview
- Memory Organization
- Memory Capacity
- Memory Interleaving

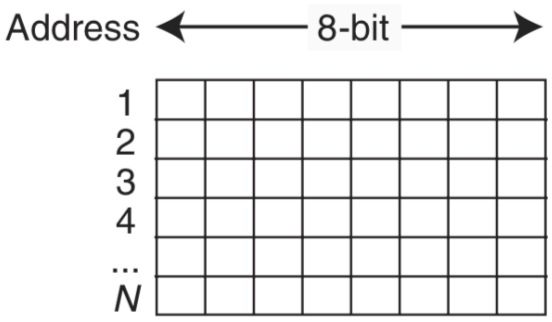
Overview

- The main memory is the central storage unit in a computer system.
- The principle technology used for the main memory is based on semiconductor integrated circuits (RAM) either *static* or *dynamic* operating modes.

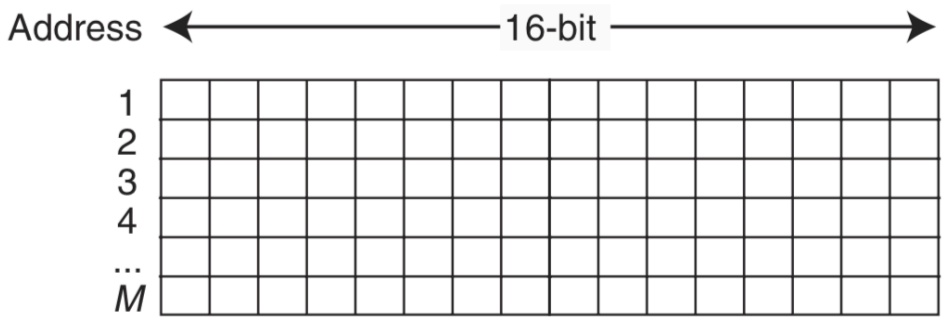
- Most of the main memory in a general-purpose computer is made up of RAM integrated circuit, but a portion of the memory may be constructed with ROM.

6 Memory Organization

- We can view memory as a matrix of bits.
- Each row has a length typically equivalent to the *word size* of the machine.
- Each row (more commonly referred to as a *memory location*) has a unique address; memory addresses usually start at zero (0) and progress downward.



N 8-Bit Memory Locations



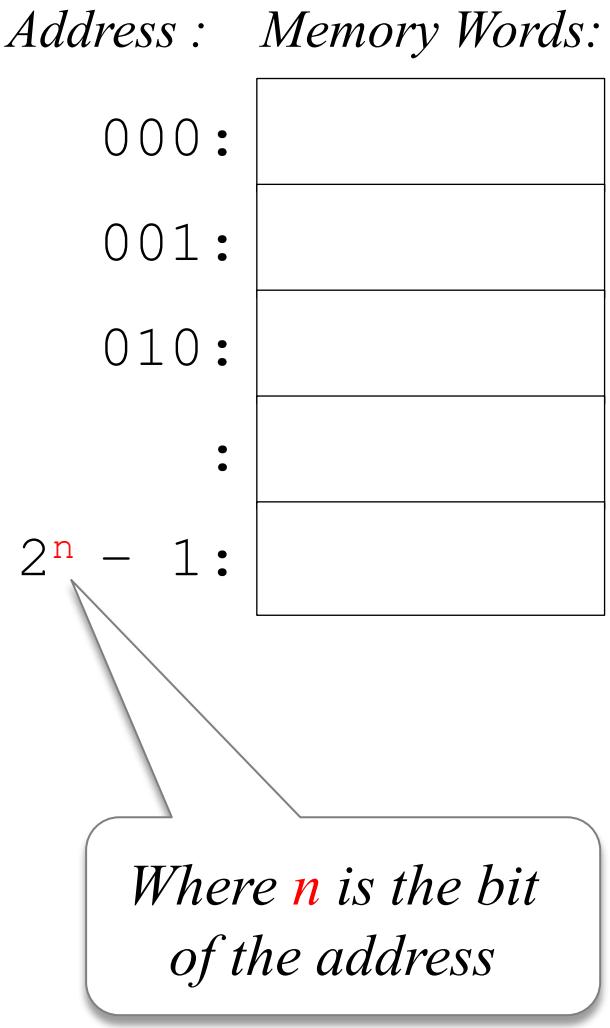
M 16-Bit Memory Locations

Memory Location

- Each part of memory has a separate memory location, which can be referred to using a memory address.
- Number of bits for an address uniquely access to a memory location.

$$No\ of\ bits = \frac{\log\left(memory\ capacity\right)}{\log 2}$$

$$No\ of\ locations = 2^{(No.of\ bit\ in\ the\ address)}$$

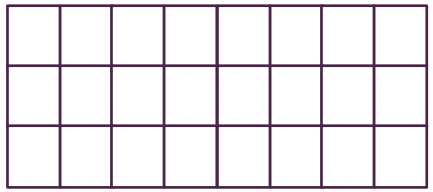


Memory Capacity

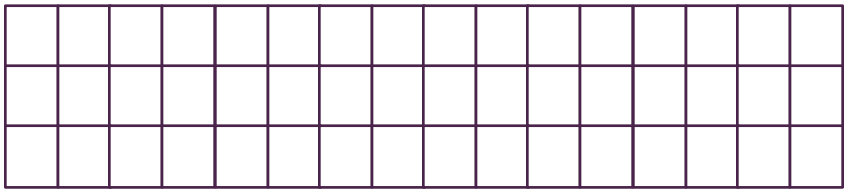
- Memory capacity is measured in bits:

total no. of memory locations (words) × size of memory word

- **Examples**: 3 words (locations/rows)



(a)



(b)

Capacity
= 3 words x 8 bits
= 24 bits

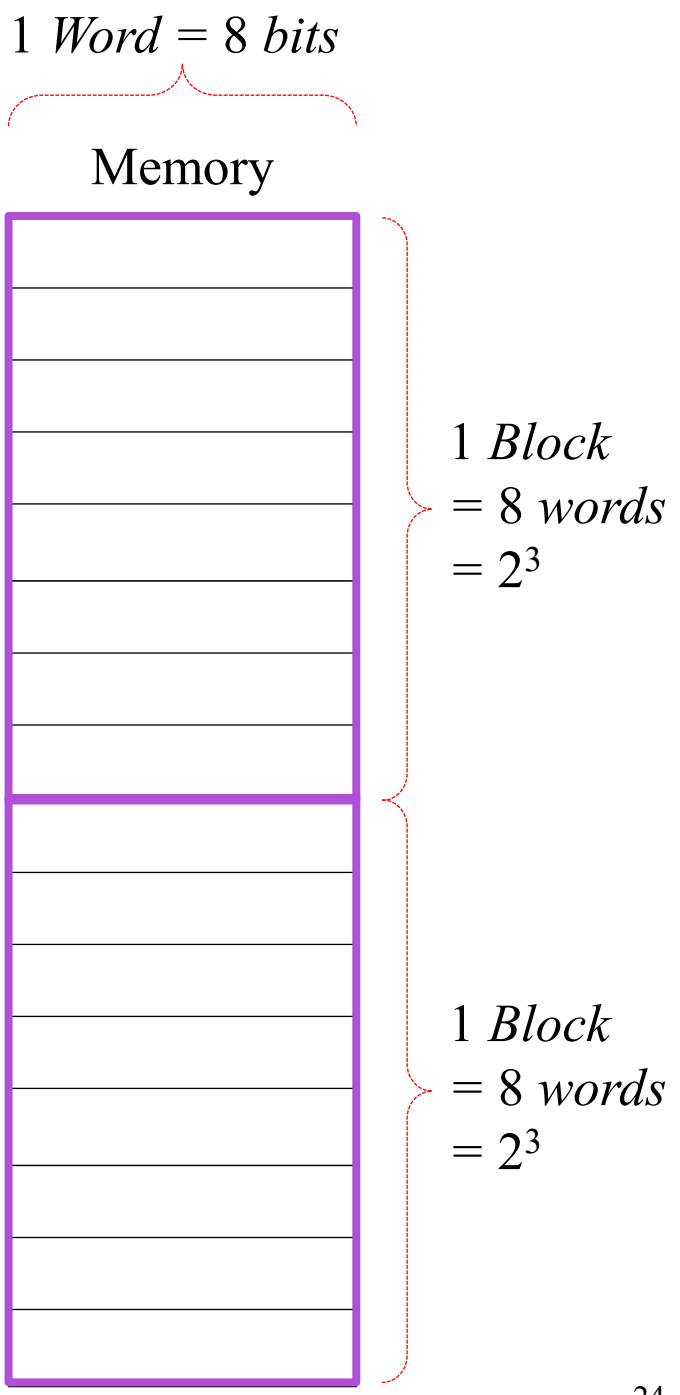
Capacity
= 3 words x 16 bits
= 48 bits

Example 1:

Main memory is divided into blocks. The memory word is 8 bit and the size of a block is 8 words.

(a) What is the capacity of the main memory, if the total number of blocks in the memory is 128?

(b) How many blocks in the main memory if the memory capacity is 32 Kbit?



Solution :

The memory word is 8 bit and the size of a block is 8 words

Q1: What is the **capacity of the main memory**, if the total number of blocks in the memory is 128?

1 Memory capacity

= total no. of memory locations (words) X size of memory word

= 1024 x 8 bit

= 8192 bits

= (8192/1024) Kbits = 8 Kbits

→ Total of memory locations (words)

= total no. of memory blocks X no. of memory word

= 128 block x 8 word = 1024

The memory word is 8 bit and the size of a block is 8 words

Q2: How many **blocks in the main memory** if the memory capacity is 32 Kbit

1 Memory capacity

= Total no. of memory locations (TML) X size of memory word

➔

32Kbit = TML X 8 bit

TML = 32Kbit / 8 bit = (32 x 1024)/8 = **4096 words**

→ Total no. of memory locations (words)

= Total no. of memory blocks X no. of memory word

➔

4096 words = total no. of memory blocks x 8

no. of memory blocks = 4096/8

= **512 blocks**

Memory Interleaving

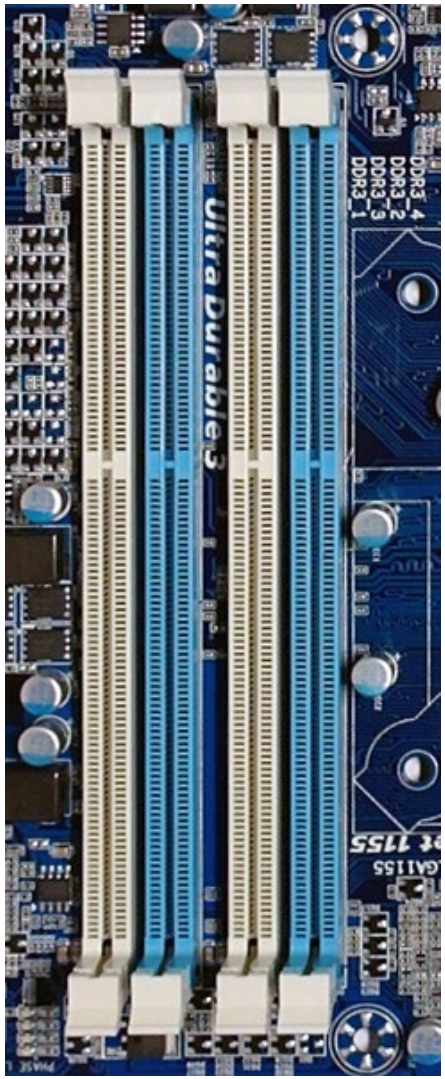
- Main memory (RAM) module is usually composed of a collection of DRAM memory chips.
- *Memory interleaving* → splits memory across multiple memory modules (or memory banks).

A number of memory chips can be grouped together to form a memory bank

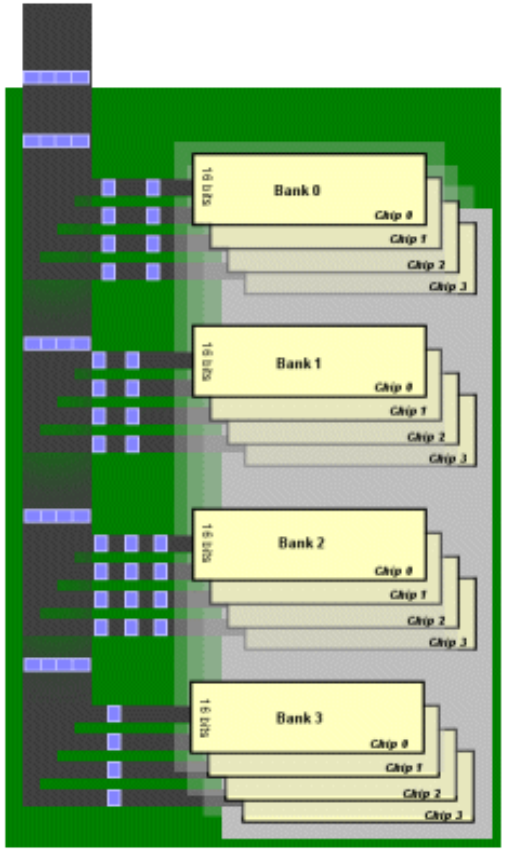
Low-Order Interleaving (LOI)	*High-Order Interleaving (HOI)*
The low-order bits of the address are used to select the bank. *e.g.* → 00000101	The high-order bits of the address are used to select the bank. *e.g.* → 10100000

Memory slots on a motherboard

Banks & Chips

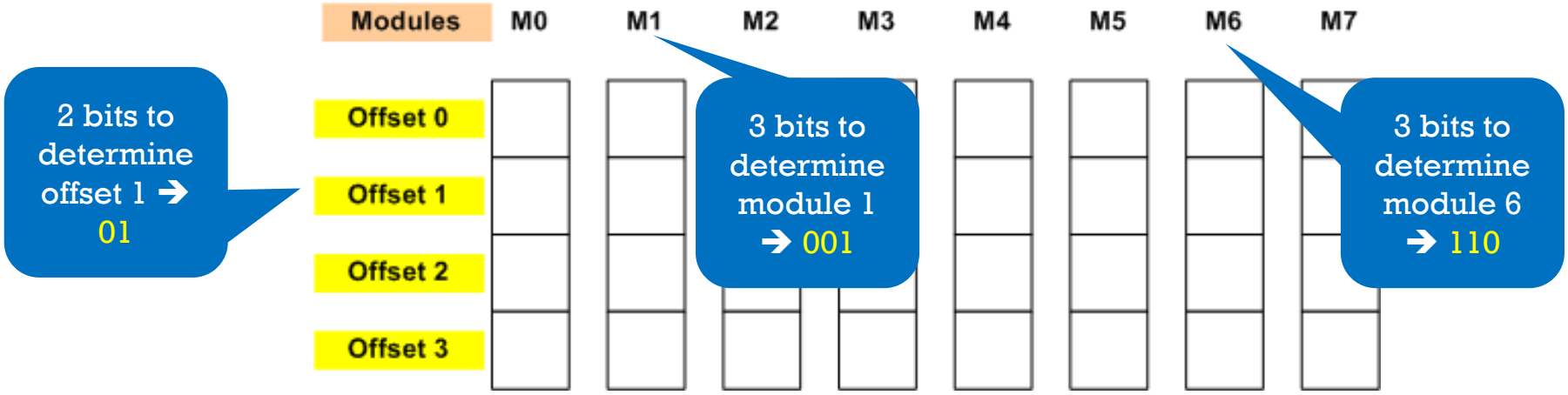






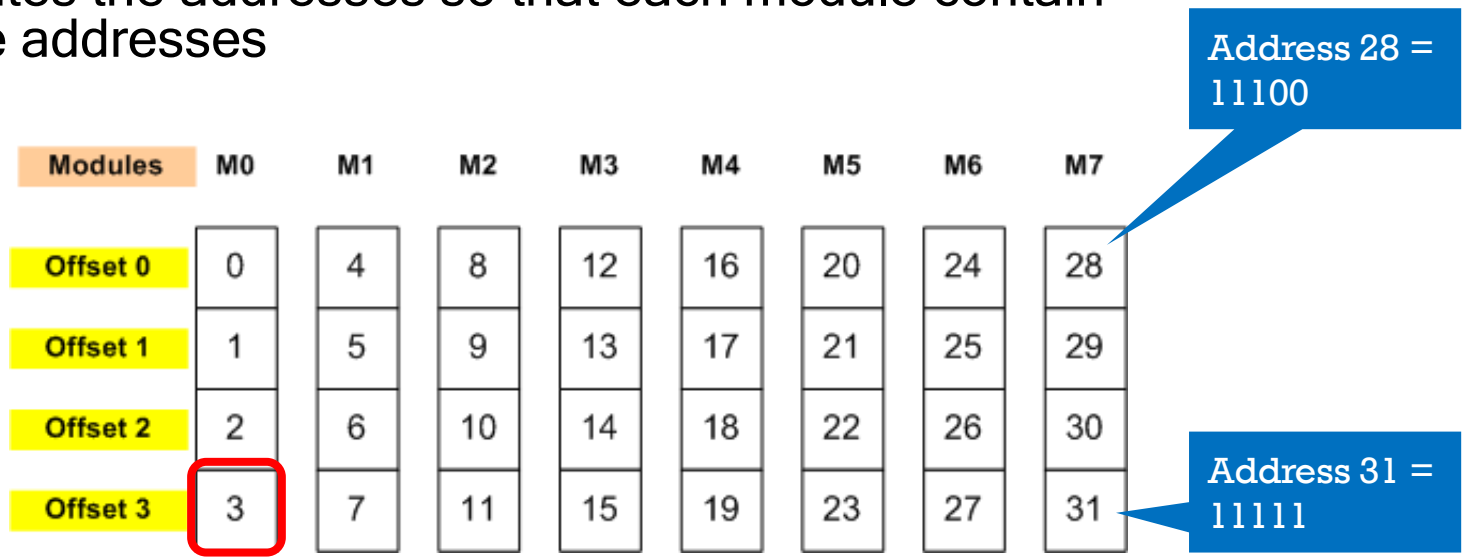
Memory interleaving on 32 bytes of address

- Assume we have a byte-addressable memory consists of 8 modules / banks of 4 bytes each.
  - This gives a total of = 32 bytes of memory (➔ 4 bytes x 8).
  - To identify each byte, we need 5 bits (➔ $2^5 = 32$)
    - 3 bits is used to determine the module ➔ there are 8 modules , so 2^3
    - 2 bits to determine the offset within the module ➔ module capacity = 4 bytes = 2^2



HOI on 32 addresses

- HOI distributes the addresses so that each module contain consecutive addresses



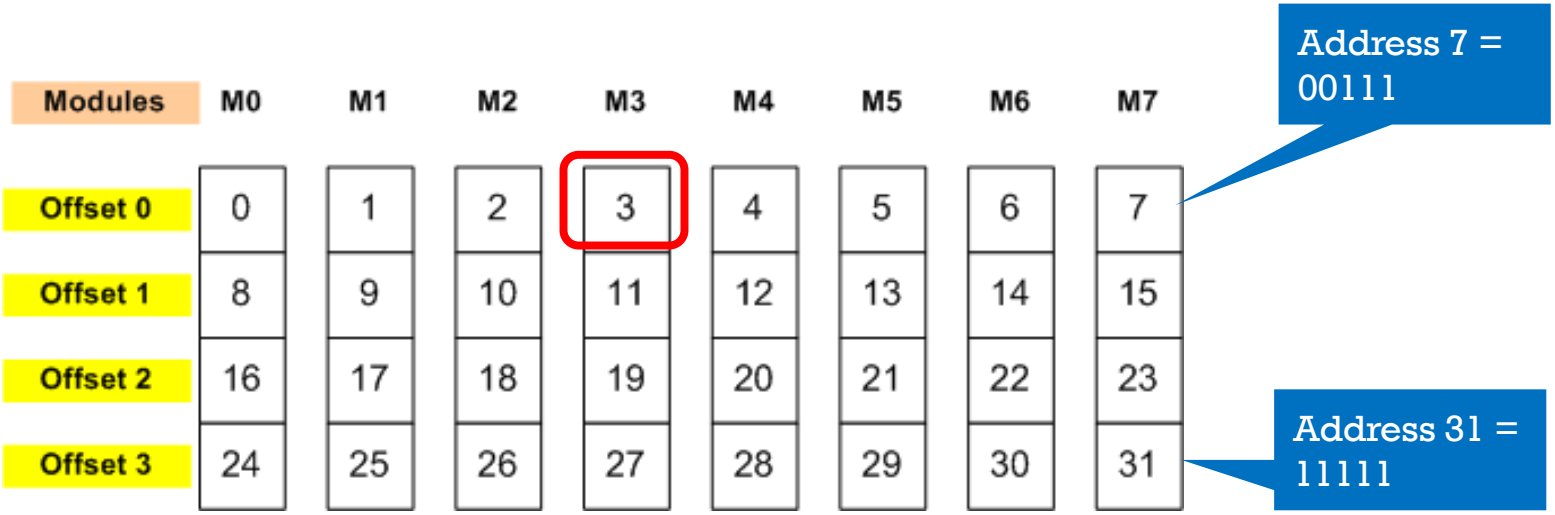
- Address 3 , in binary = 00011 (remember we need 5 bits)



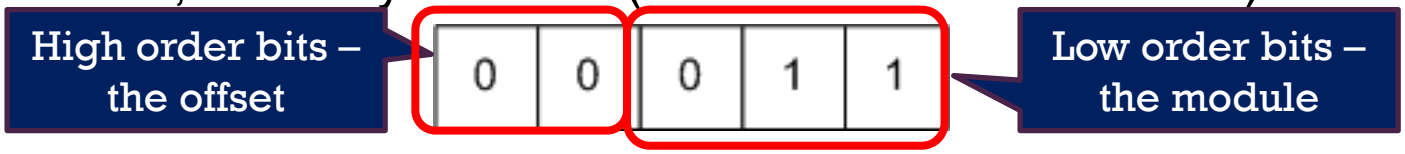
- So, address 3 is at module 0, offset 3

LOI on 32 addresses

- LOI places the consecutive address in different memory modules.



- Address 3 , in binary = 00011 (remember we need 5 bits)



- So, address 3 is at module 3, offset 0

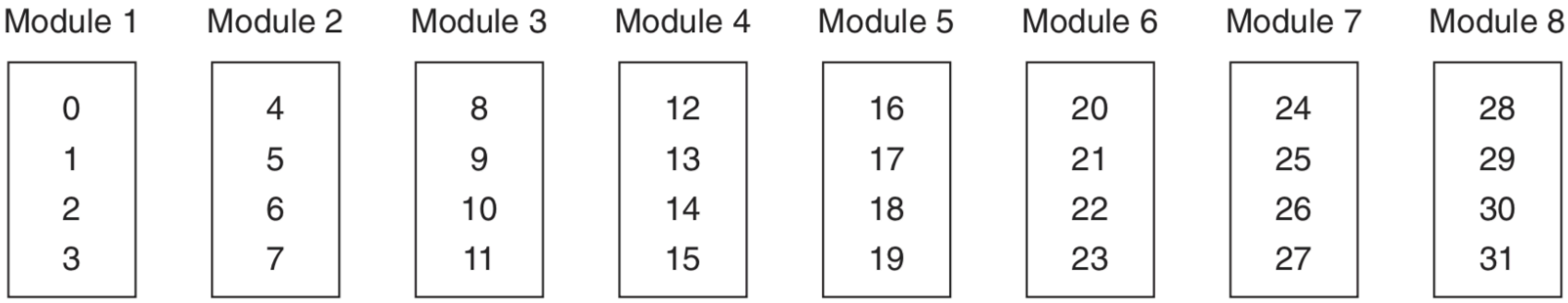


Figure: High-Order Interleaving (HOI).

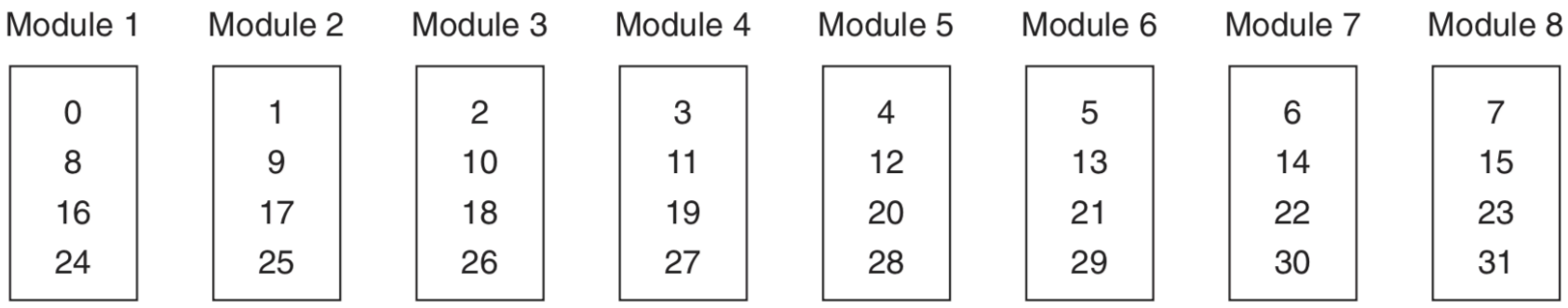


Figure: Low-Order Interleaving (LOI).

Address format:

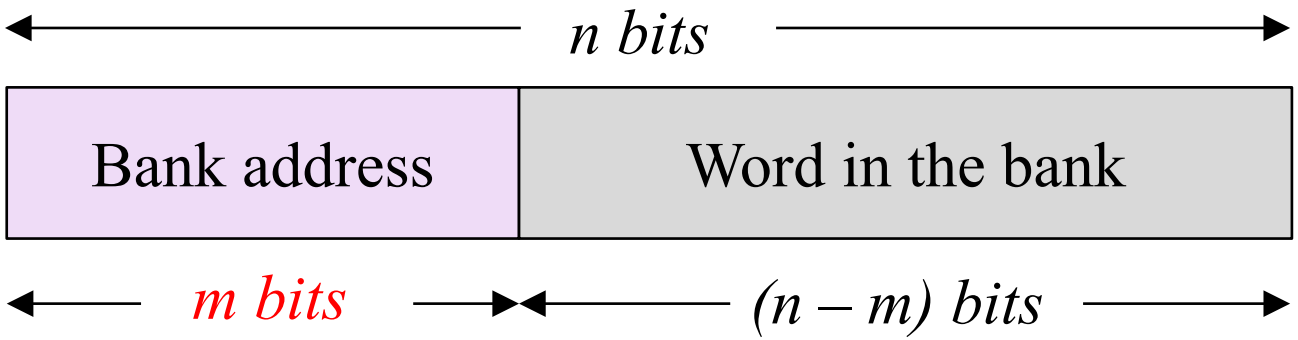


Figure: High-Order Interleaving (HOI).

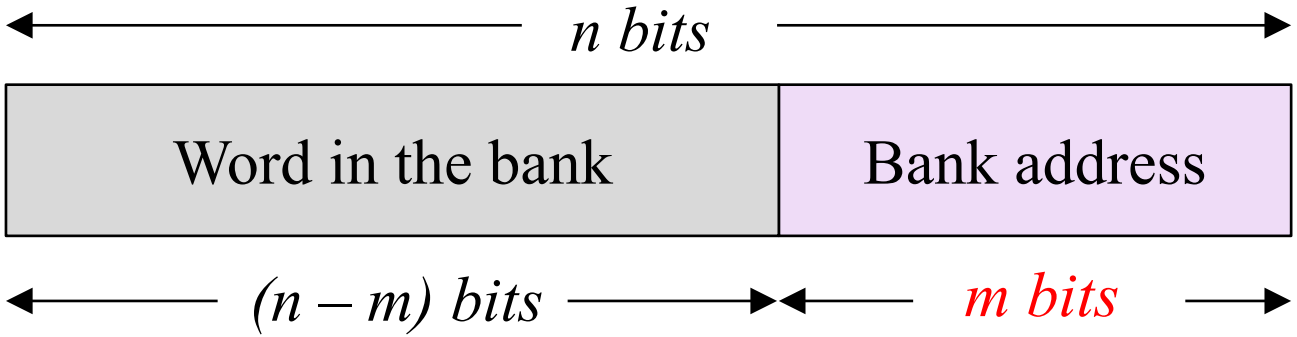
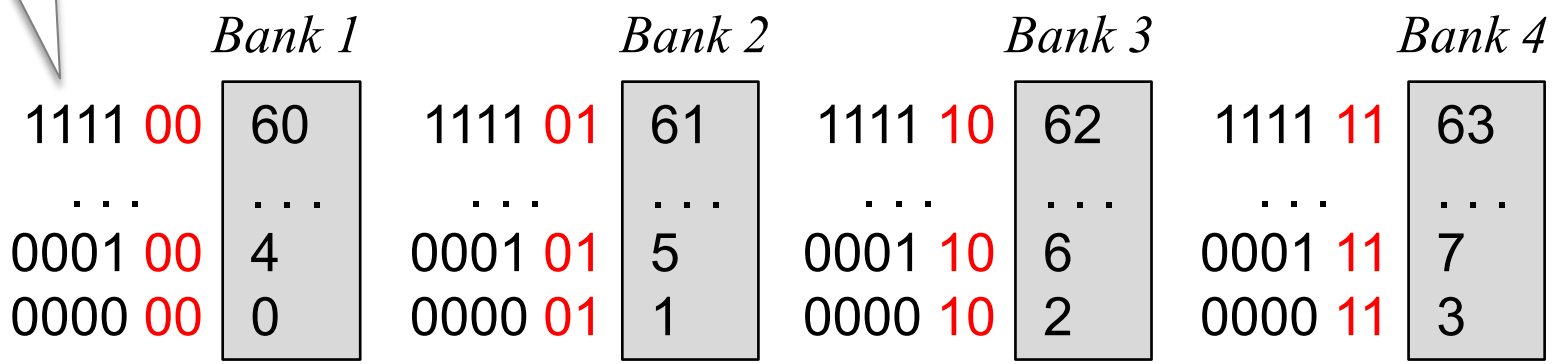


Figure: Low-Order Interleaving (LOI).

Example 3: LOI

- Memory capacity is 64 Bytes
  → 2^6 (Number of bit for memory address = 6)
- Total bank is 4
  → 2^2 (Number of bit for bank address = 2)
- Bank capacity = $2^{6-2} = 2^4$ = 16 Bytes

These 4 bits are same in all banks



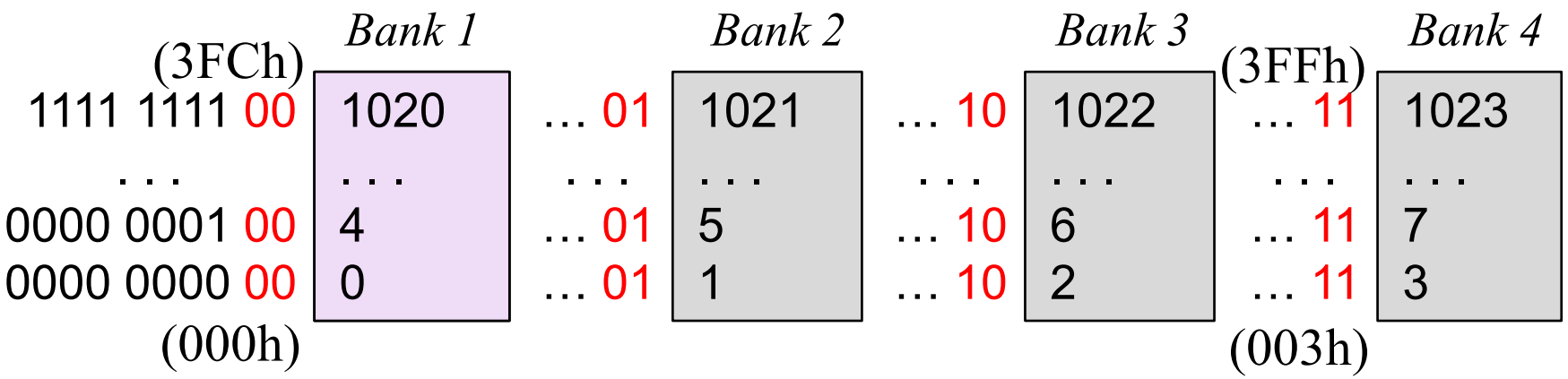
Example 4: LOI

Given a memory address as 29Ch (10 bits) and there are 4 memory banks.

Determine the memory bank address and the address of the word in the bank using LOI.

Solution:

- 4 bank → 2^2 (bit for bank address = 2)
- Bank capacity = $2^{10-2} = 2^8 = 256$ Bytes
- Memory address = 29Ch = 10 1001 1100
  - bank = 00 (i.e., Bank 1)
  - word = 10100111 (167_{10})



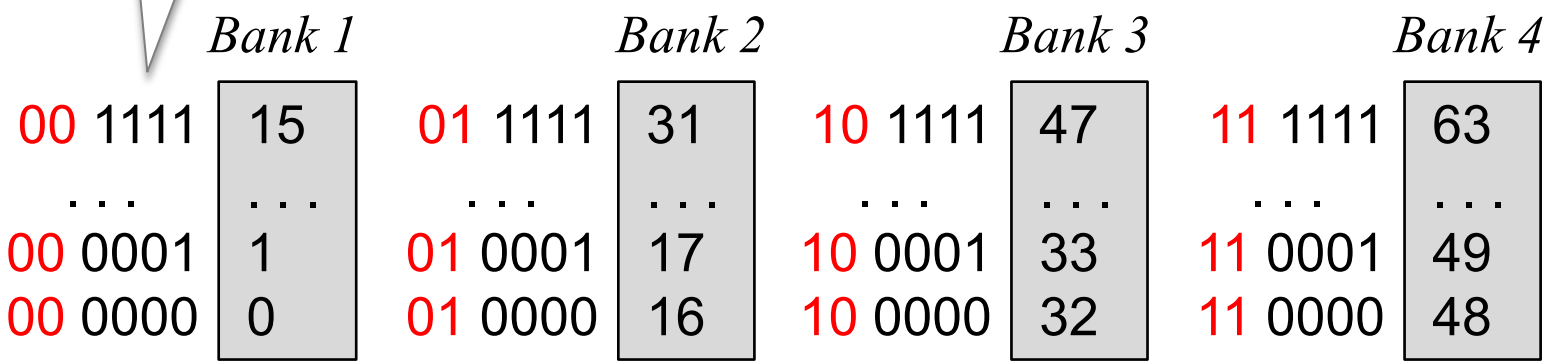
Example 5: HOI

- Memory capacity is 64 Bytes

  → 2^6 (Number of bit for memory address = 6)
- Total bank is 4

  → 2^2 (Number of bit for bank address = 2)
- Bank capacity = $2^{6-2} = 2^4 = 16$ Bytes

These 4 bits are same in all banks



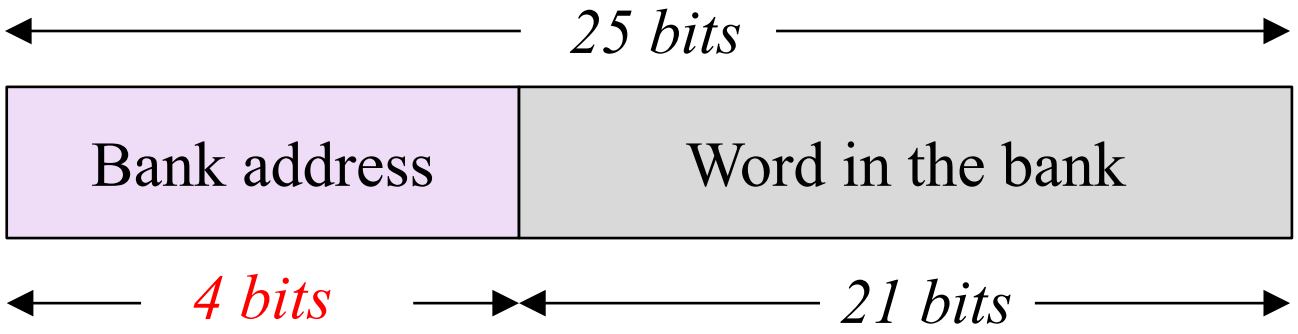
Example 6: HOI

Given a main memory has 32 *Mwords* with the size of each word is 1 *Byte*, and there are 16 memory banks.

Draw the modular memory address format if the system is implemented with HOI.

Solution:

- Main memory → 32 *Mwords* = 2^5 *Mwords* = 2^5 x 2^{20} *words* = 2^{25} (Number of bit for memory address = 25)
- 16 memory bank → 2^4 (Number of bit for bank address = 4)
- Word in the bank = 25 – 4 = 21 bits



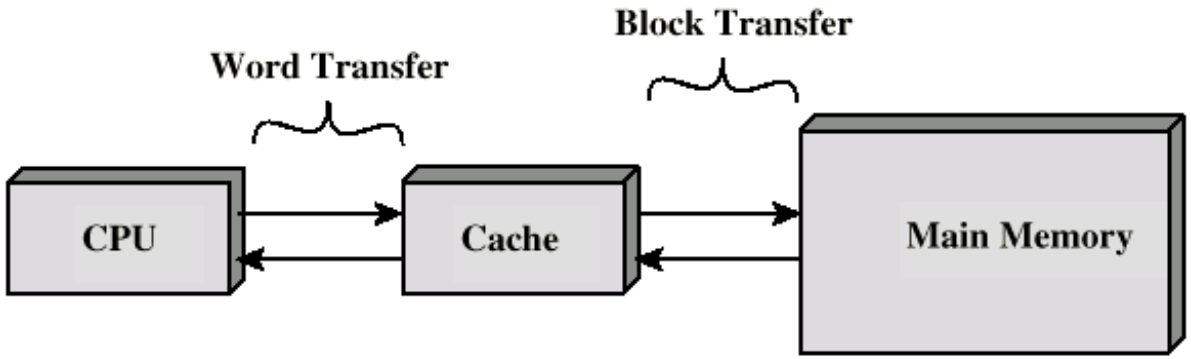
Module 6
Memory

6.1 Introduction
6.2 Main Memory
6.3 Cache Memory
6.4 Virtual Memory
6.6 Summary

- Overview
- Cache Mapping Schemes
- Replacement Policy
- Cache Performances

Cache Memory

- Small amount of fast, expensive memory.
- Sits between normal main memory (slower) and CPU.
- May be located on CPU chip or module.



- The cache contains a copy of portions of main memory.
- Unlike main memory, which is accessed by address, cache is typically accessed by content;
  - hence, it is often called content addressable memory.

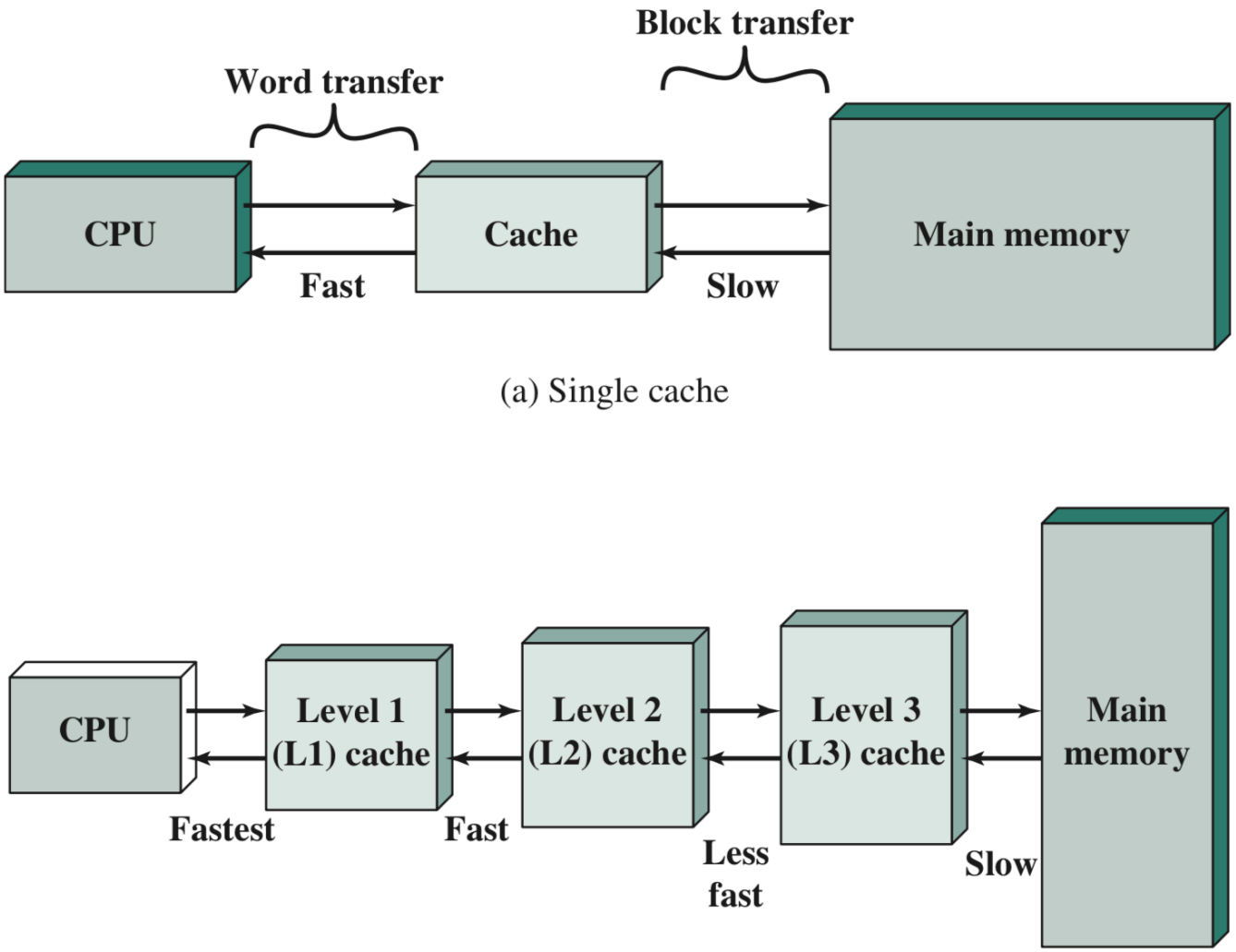
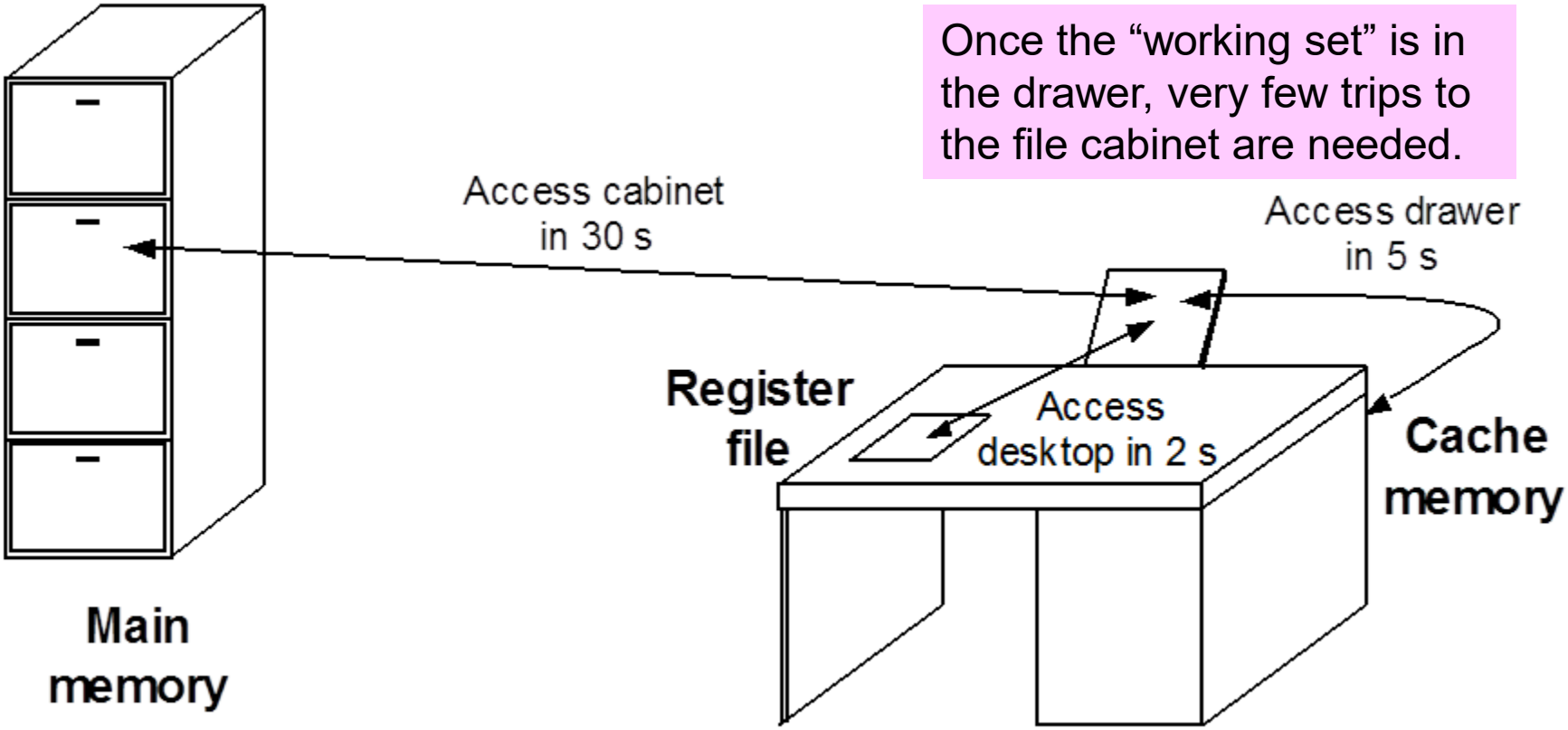


Figure: Cache and Main Memory

Analogy: Office Desk

=> *Desktop*, *Drawer*, and *File Cabinet*



Items on a desktop (register) or in a drawer (cache) are more readily accessible than those in a file cabinet (main memory).

Principle of Locality

- In practice, most programs exhibit locality, which the cache can take advantage of.
- The principle of temporal locality says that if a program accesses one memory address, there is a good chance that it will access the same address again.
- The principle of spatial locality says that if a program accesses one memory address, there is a good chance that it will also access other nearby addresses.

Temporal Locality in Programs and Data

- **Programs**

  Loops are excellent examples of temporal locality in programs.

  - The loop body will be executed many times.
  - The computer will need to access those same few locations of the instruction in memory repeatedly.

- **Data**

  Programs often access the same variables over and over, especially within loops.

Spatial Locality in Programs and Data

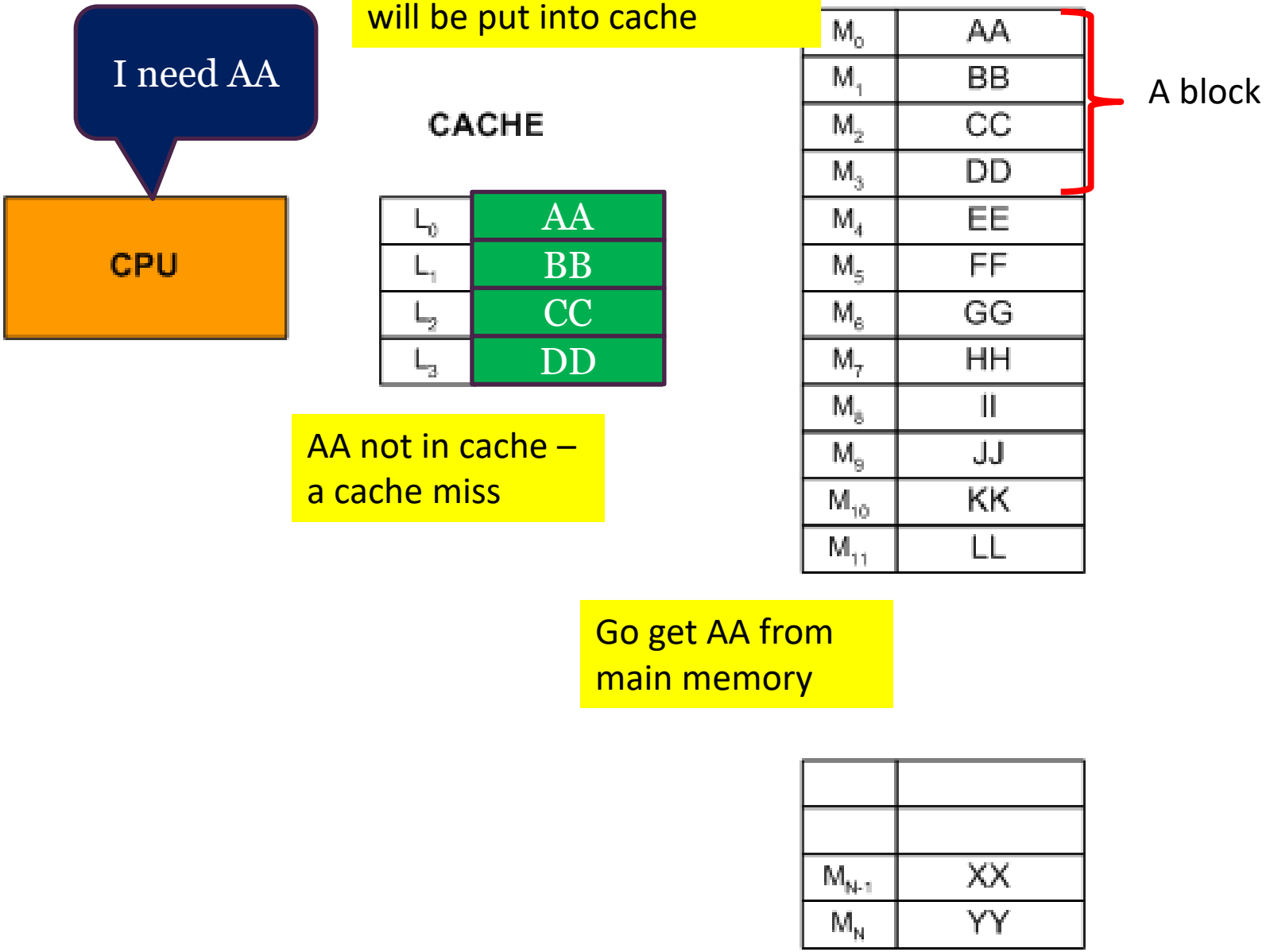
- Programs
  - Nearly every program exhibits spatial locality, because instructions are usually executed in sequence
  - if CPU execute an instruction at memory location *i, then it will probably also execute the next instruction, at* memory location *i+1.*
  - Code fragments such as *loops* exhibit *both temporal and spatial locality.*

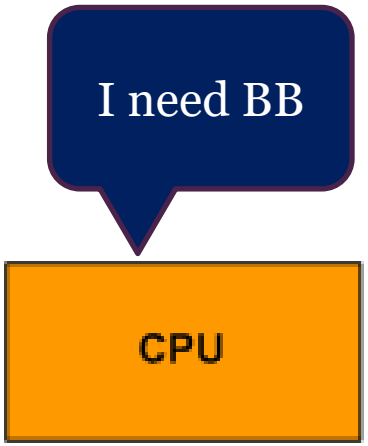
Spatial Locality in Programs and Data

- Data
  - Programs often *access data* that is stored contiguously.
  - *Arrays* are stored in memory contiguously.
  - The individual *fields of a record* or *object* like employee are also kept contiguously in memory.

How is data copied into cache?

- Main memory and cache are both divided into the same size blocks.
- When a memory address is generated, cache is searched first to see if the required word exists there.
- When the requested word is not found in cache, the entire main memory block in which the word resides will be loaded into cache.
- This scheme is successful because of the principle of locality;
  - if a word was just referenced, there is a good chance words in the same general vicinity will soon be referenced as well.


I need AA
CPU
A block from main memory will be put into cache
CACHE
L0 AA
L1 BB
L2 CC
L3 DD
AA not in cache – a cache miss
M0 AA
M1 BB
M2 CC
M3 DD
M4 EE
M5 FF
M6 GG
M7 HH
M8 II
M9 JJ
M10 KK
M11 LL
A block
Go get AA from main memory
MN-1 XX
MN YY



CACHE

L_0	AA
L_1	BB
L_2	CC
L_3	DD

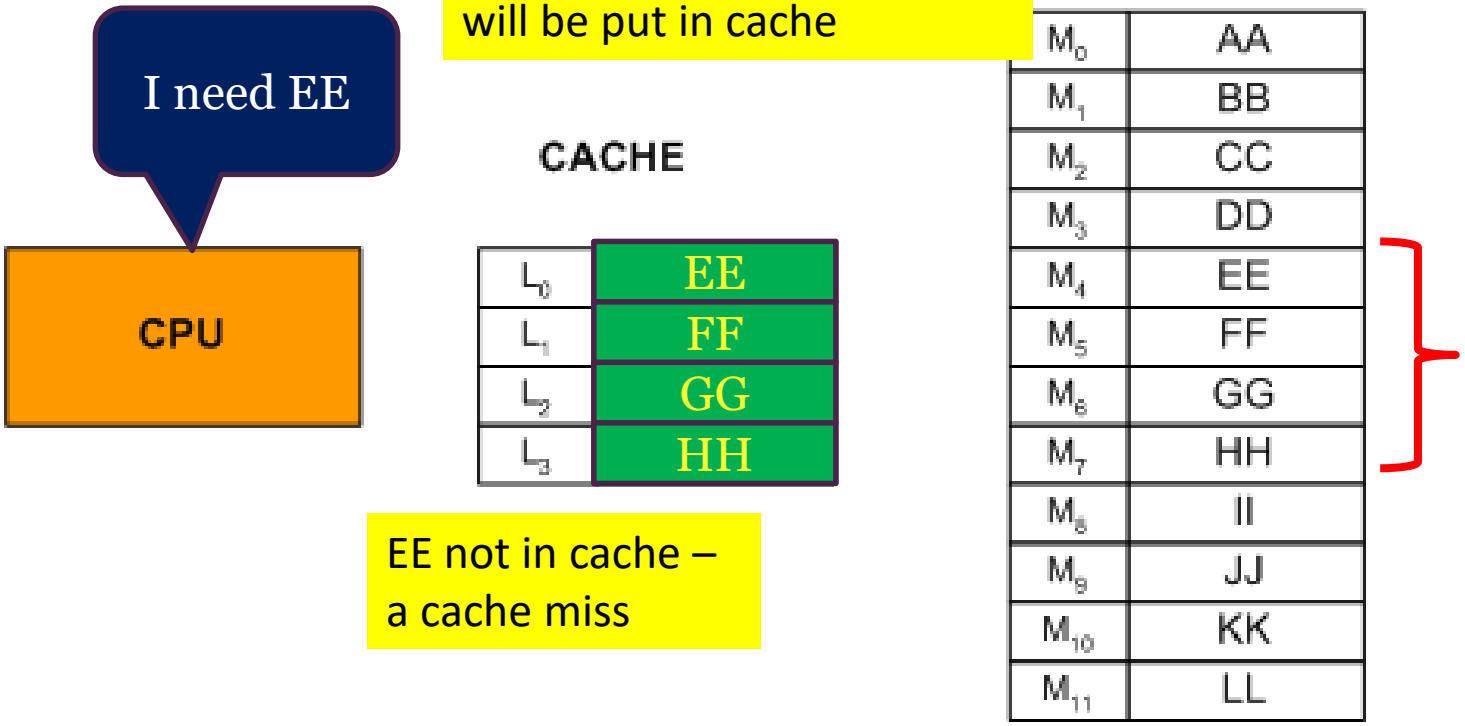
M_0	AA
M_1	BB
M_2	CC
M_3	DD
M_4	EE
M_5	FF
M_6	GG
M_7	HH
M_8	II
M_9	JJ
M_{10}	KK
M_{11}	LL

A block

BB is in cache – a cache hit

Principle of locality is at work here.

M_{N-1}	XX
M_N	YY


I need EE
CPU
A block from main memory will be put in cache
CACHE
L0 EE
L1 FF
L2 GG
L3 HH
M0 AA
M1 BB
M2 CC
M3 DD
M4 EE
M5 FF
M6 GG
M7 HH
M8 II
M9 JJ
M10 KK
M11 LL
EE not in cache – a cache miss
Go get EE from main memory
MN-1 XX
MN YY

Mapping functions

- Because there are fewer cache lines than main memory blocks, an algorithm is needed for mapping main memory blocks into cache lines.
- There is also a need for a way to determine which main memory block currently occupies a cache line.
- The choice of the mapping function dictates how the cache is organized.

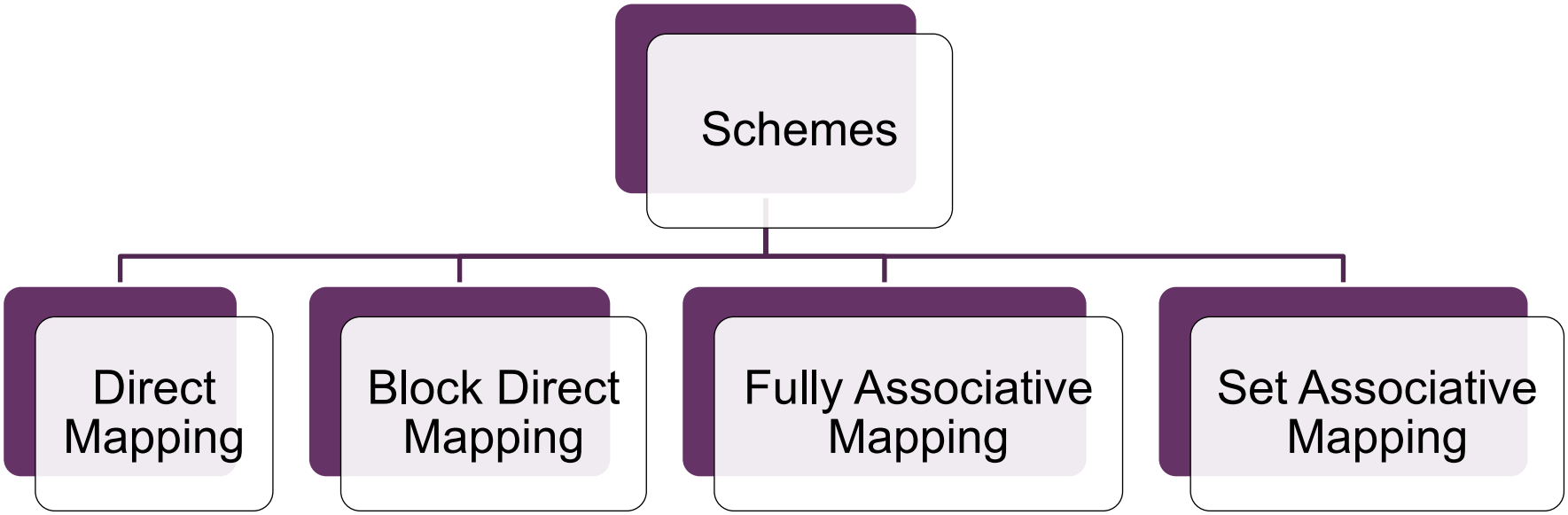


Figure: Cache mapping schemes

Linda Null and Julia Lobur (2003). *The Essentials of Computer Organization and Architecture.* United States: Jones and Bartlett Publishers. p.240

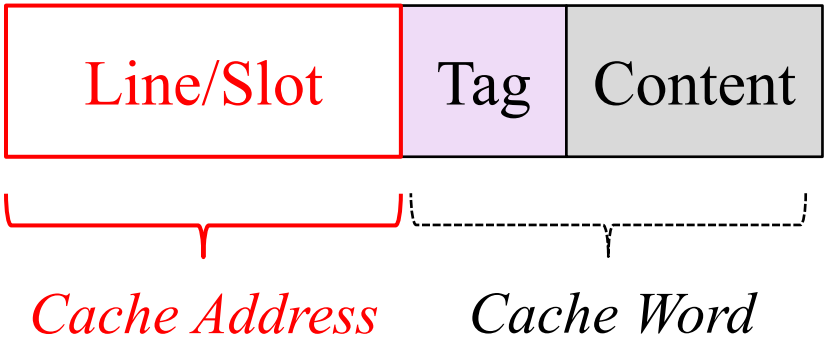
(a) Direct Mapping

- The simplest technique.
- *A cache structure in which each memory location is mapped to exactly one location in the cache.
- Address formats:

Main memory address :



Cache Memory :



*Patterson, D.A. and Hennessy, J.L. (2014). *Computer Organization and Design: The Hardware/Software Interface (5th Edition)*. United States: Elsevier, p.384

Example 7: Direct Mapping.

A main memory contains 8 words while the cache has only 4 words. Using direct address mapping, identify the fields of the main memory address.

Solution :

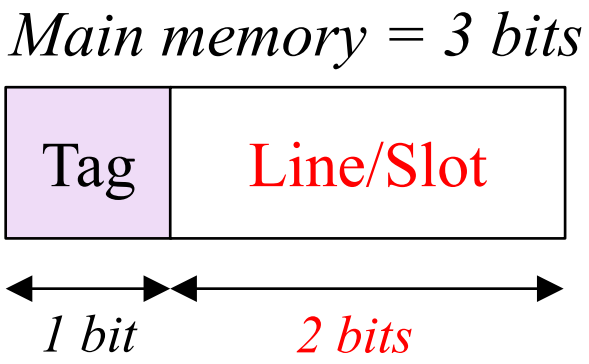
- Total memory words = 8 = 2^3

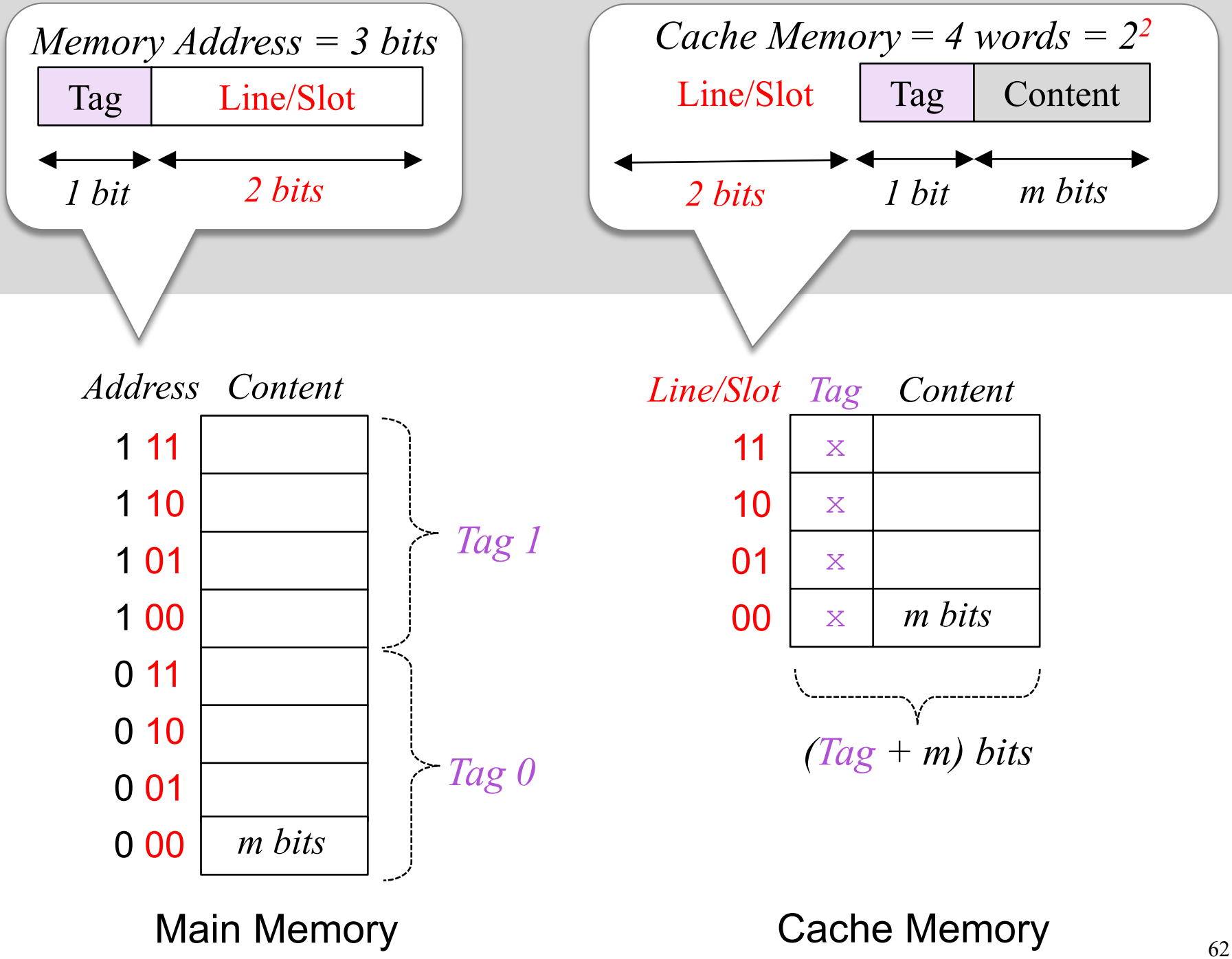
  → Require 3 bits for main memory address.

- Total cache words = 4 = 2^2

  → Require 2 bits for cache address (*Line/Slot*)

- Tag size = 3 – 2 = 1 bit




Memory Address = 3 bits
Tag
Line/Slot
1 bit
2 bits
Cache Memory = 4 words = 2^2
Line/Slot
Tag
Content
2 bits
1 bit
m bits
Address
Content
1 11
1 10
1 01
1 00
0 11
0 10
0 01
0 00
m bits
Tag 1
Tag 0
Line/Slot
Tag
Content
11
10
01
00
x
x
x
x
m bits
(Tag + m) bits
Main Memory
Cache Memory

Example 8: Direct Mapping.

A main memory contains 32 words while the cache has only 8 words. Using direct address mapping, identify the fields of the main memory address.

Solution :

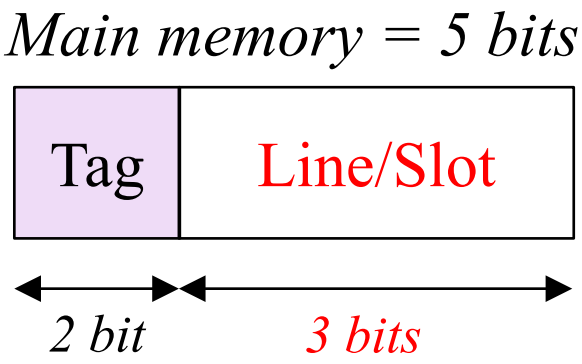
- Total memory words = 32 = 2^5

  → Require 5 bits for main memory address.

- Total cache words = 8 = 2^3

  → Require 3 bits for cache address (*Line/Slot*)

- Tag size = 5 – 3 = 2 bits



Example 9: Direct Mapping.

Size of cache memory is 64*K word* and the size of main memory is 64*M* × 8 *bit* word. Determine the word size of main memory, cache and the main memory address format.

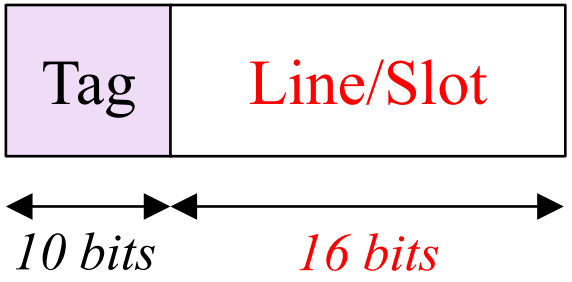
Solution :

- Total words in main memory = $64M = 2^6 \times 2^{20} = 2^{26}$

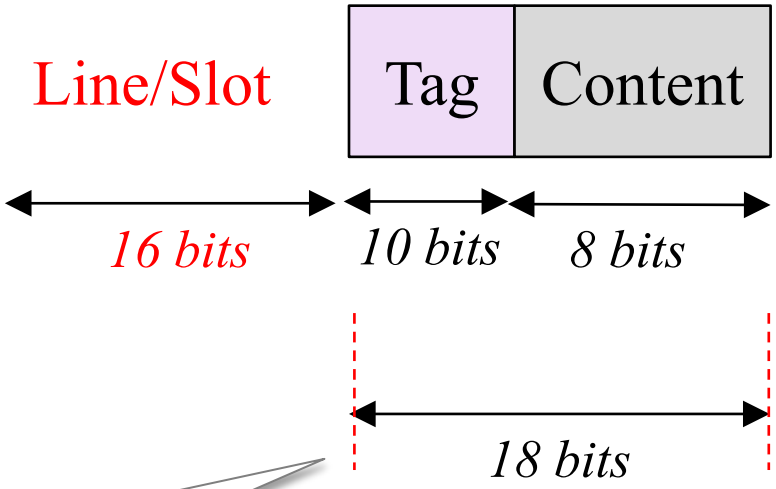
  → Require 26 bits for main memory address.
- Total cache words = $64K = 2^6 \times 2^{10} = 2^{16}$ → 16 bits for *Line/Slot*
- Tag size = 26 – 16 = 10 bits
- Size of main memory word = 8 *bits*
- Size of cache word = Tag + (No. words in cache × size)

  = 10 + (1 × 8) = 18 *bits*

Main memory = 26 bits



Cache Memory = 64K = 2^{16}



Cache word
= (Tag + size of content)
= 10 + 8 = 18 bits

Example 10:

A main memory contains 16 words while the cache has only 4 words. Using direct address mapping, identify the fields of the main memory address.

Solution :

- Total main memory words = 16 = 2^4

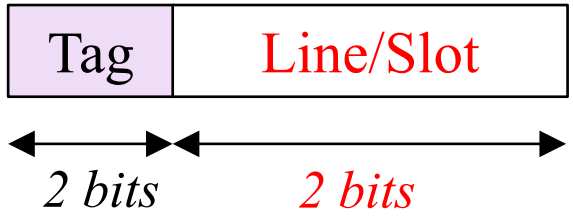
  → Require 4 bits for main memory address.

- Total cache words = 4 = 2^2

  → Require 2 bits for cache address (*Line/Slot*)

- Tag size = 4 – 2 = 2 bits

Main memory = 4 bits



Dec/Hex.	*Line/Slot*	*Tag*	*Content*
3	11	xx	
2	10	xx	
1	01	xx	
0	00	xx	

Cache Memory

… consider the main memory with contents, observe the line/slot number for both cache & main memory

Memory address:
→ (*Tag* | *Line/Slot*)

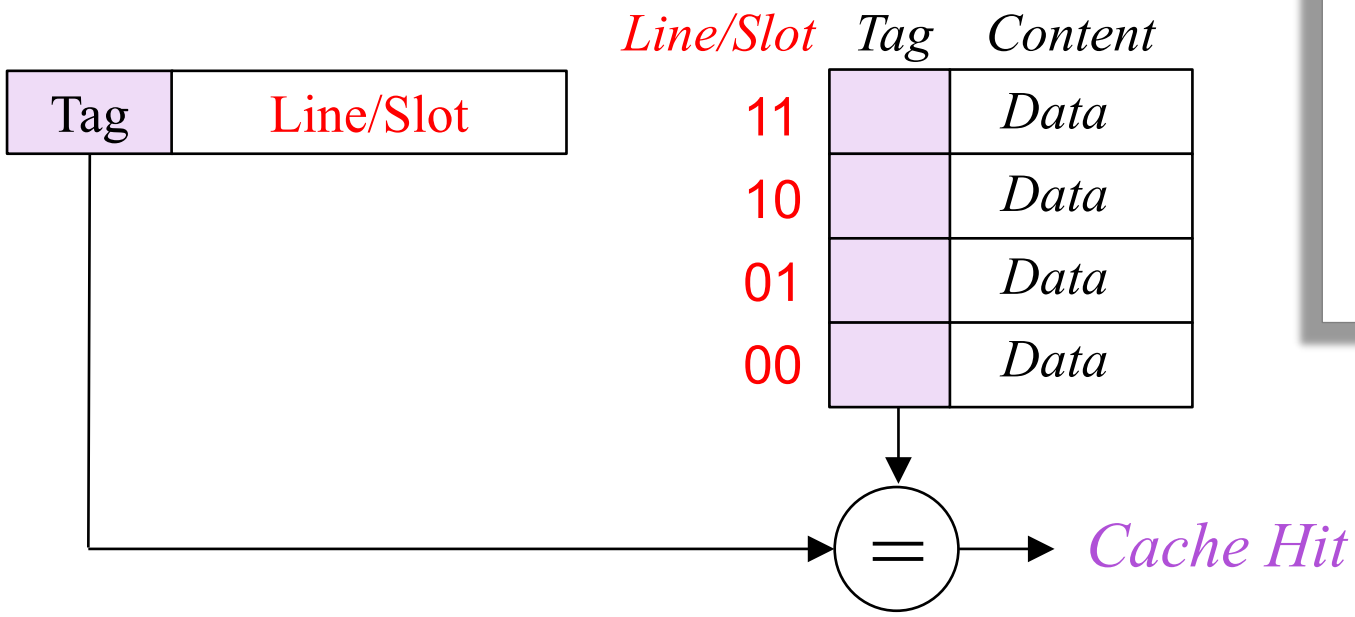
Dec/Hex.	*Line/Slot*	*Tag*	*Content*
3	11	xx	xx
2	10	xx	xx
1	01	xx	xx
0	00	xx	xx

Cache Memory

	Address	*Content (Hex)*
F	11 11	11
E	11 10	10
D	11 01	01
C	11 00	A1
B	10 11	F5
A	10 10	F4
9	10 01	F3
8	10 00	F1
7	01 11	FF
6	01 10	91
5	01 01	10
4	01 00	19
3	00 11	13
2	00 10	02
1	00 01	01
0	00 00	EE
Hex.		

Main Memory

Cache Hit and Miss



*Cache Miss

A request for data from the cache that cannot be fulfilled because the data is not present in the cache.

*Patterson, D.A. and Hennessy, J.L. (2014). *Computer Organization and Design: The Hardware/Software Interface (5th Edition)*. United States: Elsevier, p.392

Example 11:

Cache hit and miss operation.

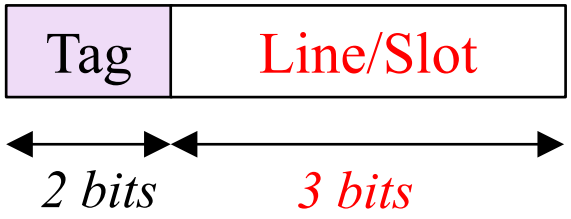
(Read Process)

A main memory contains 32 words while the cache has only 8 words. Using direct address mapping, identify the fields of the main memory address.

Solution :

- Total main memory words = 32 = 2^5 → 5 bits
- Total cache words = 8 = 2^3 → 3 bits (*Line/Slot*)
- Tag size = 5 – 3 = 2 bits

Main memory = 5 bits



Memory address:
→ (*Tag* | *Line/Slot*)

… the main memory with contents.

Dec/Hex.	*Line/Slot*	*Tag*	*Content*
7	111	xx	xx
6	110	xx	xx
5	101	xx	xx
4	100	xx	xx
3	011	xx	xx
2	010	xx	xx
1	001	xx	xx
0	000	xx	xx

Cache Memory

Hex.	*Dec*	*Address*	*Content (Hex)*
1F	31	11 111	11
1E	30	11 110	10
1D	29	11 101	01
1C	28	11 100	A1
1B	27	11 011	F5
1A	26	11 010	F4
19	25	11 001	F3
18	24	11 000	F1
17	23	10 111	FF
16	22	10 110	91
15	21	10 101	10
14	20	10 100	19
13	19	10 011	13
12	18	10 010	02
11	17	10 001	01
10	16	10 000	EE
…	…	…	…
3	3	00 011	A3
2	2	00 010	A1
1	1	00 001	AB
0	0	00 000	01

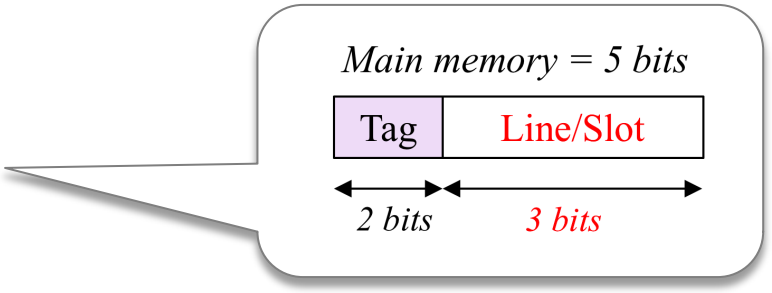
Main Memory

Based on the example, show the contents of the cache as it responds to a series of request (decimal address):

22, 26, 22, 26, 16, 3, 16, 18

Generated Address			Format Address	
Address		Content	Tag	Line/Slot
(Dec)	*(Binary)*	*(Hex)*		
22	10110	91	10	110
26	11010	F4	11	010
22	10110	91	10	110
26	11010	F4	11	010
16	10000	EE	10	000
3	00011	A3	00	011
16	10000	EE	10	000
18	10010	02	10	010

Main Memory



Based on the example, show the contents of the cache as it responds to a series of request (decimal address):

22, 26, 22, 26, 16, 3, 16, 18

Address		Content
(Dec)	*(Binary)*	*(Hex)*
22	10110	91
26	11010	F4
22	10110	91
26	11010	F4
16	10000	EE
3	00011	A3
16	10000	EE
18	10010	02

Format Address		
Line/Slot	Tag	Content *(Hex)*
111	xx	xx
110	10	91
101	xx	xx
100	xx	xx
011	00	A3
010	~~11~~ 10	~~F4~~ 02
001	xx	xx
000	10	EE

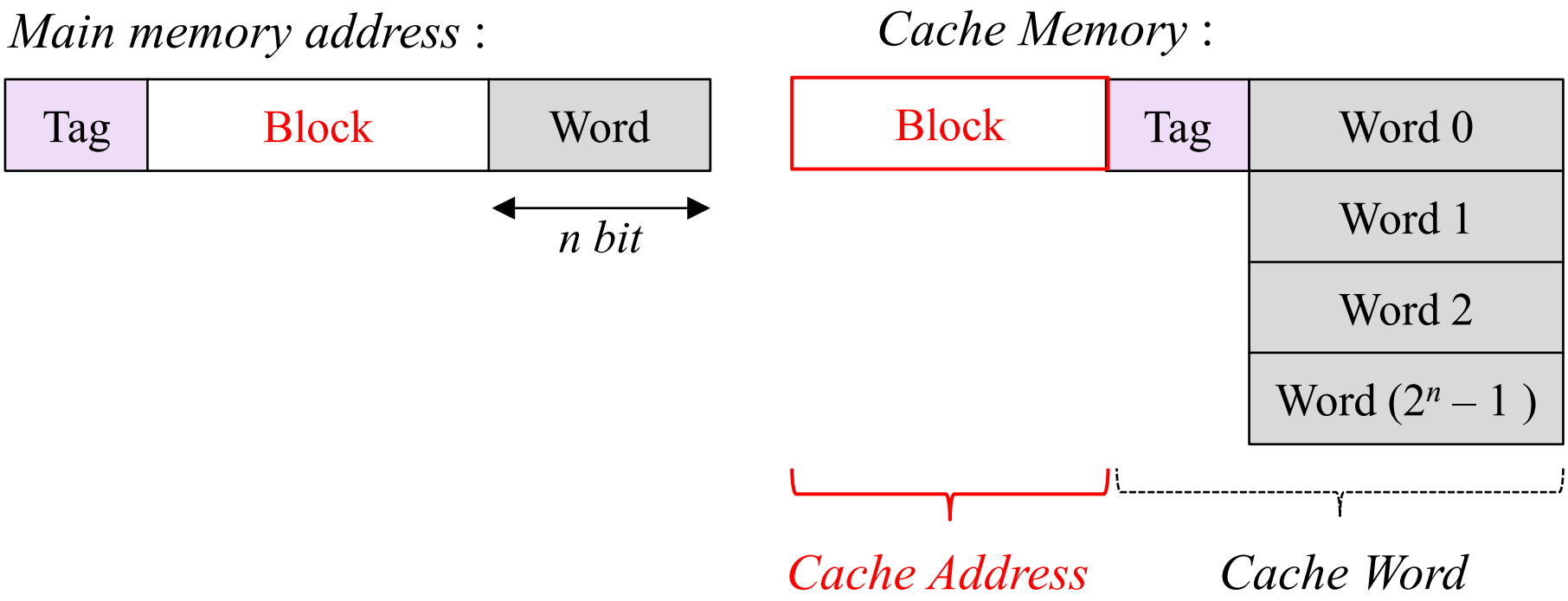
Cache Memory

Cache memory after updating

(b) Block Direct Mapping

- The simplest technique of direct mapping can maps each block of main memory into only one possible cache line.

- Address formats:



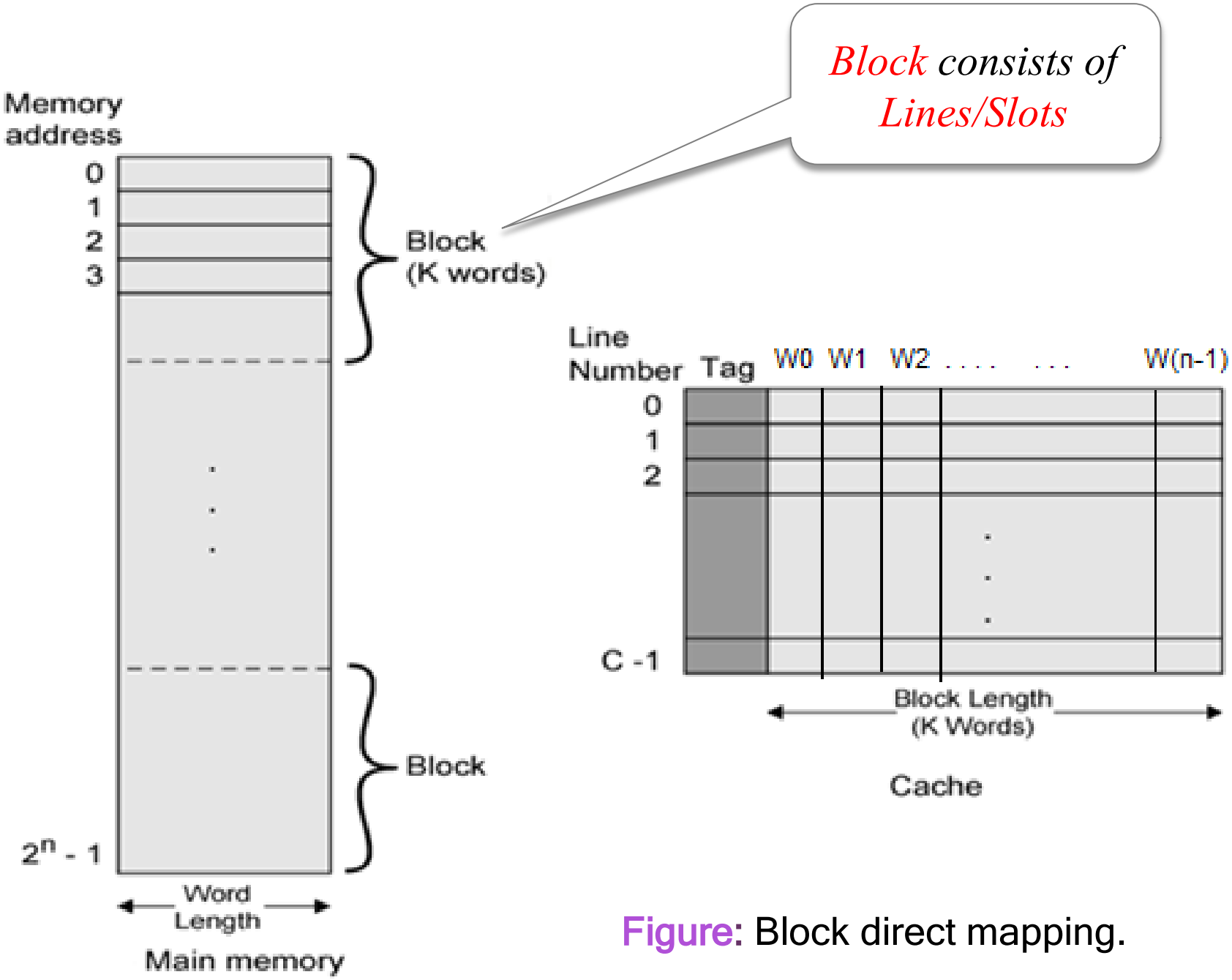
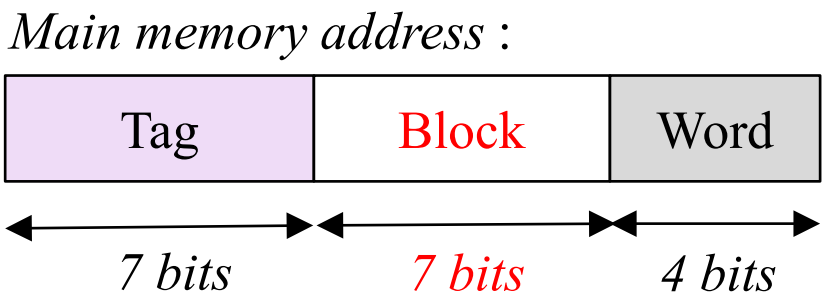


Figure: Block direct mapping.

Example 12:



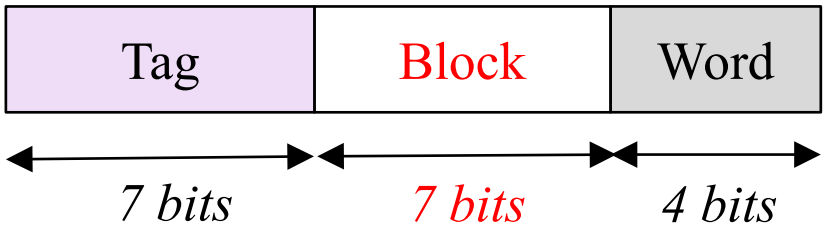
Given the above main memory address format using block direct mapping, if each main memory word is 8 bits, calculate:

a) The main memory capacity.

b) Total cache words.

c) The size of cache words.

Main memory address :



Solution :

a) Total main memory bits = 7 + 7 + 4 = 18 bits

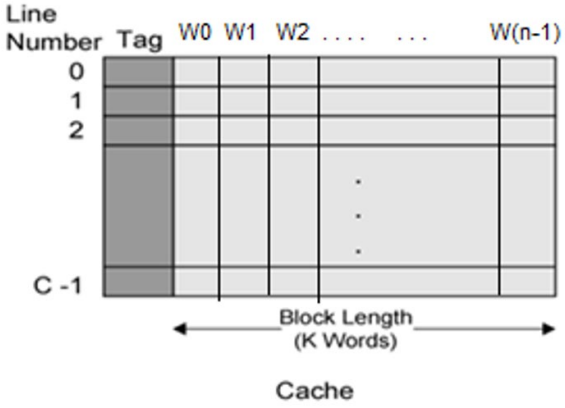
→ Total memory words = 2^8 x 2^{10} = 256 *Kword*

→ Total memory capacity = 256 *Kword x* 8 *bits*

= 2 *Mbit*

b) Block = 7 bits

→ Total cache words = 2^7 → 128 words (per *Block*)



c) Total words = 2^4 = 16 words

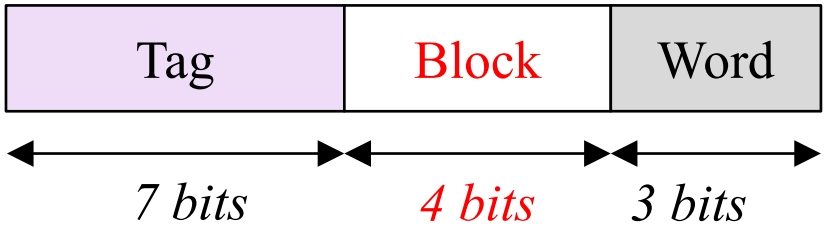
→ Cache word size = Tag + (No. of words in cache × size)

= 7 + (16 × 8)

= 135 bits

Example 13:

Main memory address :



Consider the main memory address.

Suppose a program generates the address **1AA**.

→ In 14-bit binary, this number is: 00 0001 1010 1010

Padding with bit 0 at MSB to complete 14-bit

- ✓ The first 7 bits of the address go in the tag field.
- ✓ The next 4 bits go in the block field.
- ✓ The final 3 bits indicate the word within the block.

Main memory address for **1AA** :

0000011	0101	010

Main memory address for **1AA** :

0000011	0101	010

Main memory address :

1AB

0000011	0101	011
Tag	*Block*	*Word*

Example 14: (Cache hit)

Supposed the program generates the subsequent address 1AB.

→ 1AB : 00 0001 1010 1011

It will find the data it is looking in cache memory for in block 0101, *word* 011.

Cache Memory :

0101	0000011	000	001	010	011	100	101	110	111

Block *Tag* *Words* ($2^3 = 8$)

Main memory address :

3AB	0000111	0101	011
	Tag	*Block*	*Word*

Example 15: (Cache miss)

Supposed the program generates the address 3AB.

→ In 14-bit binary, this number is: 00 0011 1010 1011

A miss occurred (tag is different).

The tag of cache memory for block 0101, must be replaced with 0000111.

Cache Memory :

0101	0000111	000	001	010	011	100	101	110	111

Block *Tag* *Words* $(2^3 = 8)$

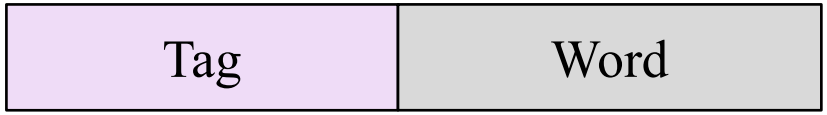
Associative Mapping



(c) Fully Associative Mapping

- Overcomes the disadvantage of direct mapping by permitting each main memory block to be loaded into any location (line/slot/block) of the cache.
- A memory address is partitioned into only two fields:

Main memory address :



- With associative mapping, there is flexibility as to which block to replace when a new block is write into the cache.

Linda Null and Julia Lobur (2003). *The Essentials of Computer Organization and Architecture.* United States: Jones and Bartlett Publishers. p.245
William Stallings (2016). *Computer Organization and Architecture: Designing for Performance* (10th Edition). United States: Pearson Education Limited, p.138
*Patterson, D.A. and Hennessy, J.L. (2014). *Computer Organization and Design: The Hardware/Software Interface (*5th Edition). United States: Elsevier, p.403

- When the cache is searched for a specific main memory block, the tag field of the main memory address is compared to all the valid tag fields in cache;

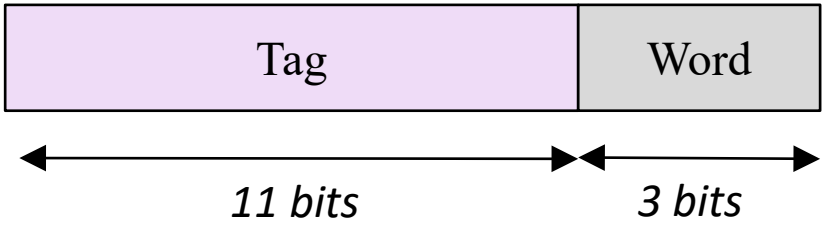
  - ✓ if a match is found, cache hit.
  - ✓ If there is no match, we have a cache miss and the block must be transferred from main memory.

Example 16:

Consider a memory configuration with 2^{14} words, a cache with 16 blocks, and a block of 8 words.

- Each block = 8 words = 2^3 → 3 bits (*Word*)
- Tag size = 14 – 3 = 11 bits

Main memory address = 14 bits

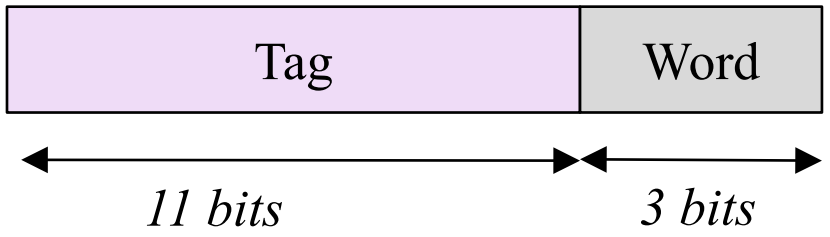


Cache Memory :



Example 17:

Main memory address :



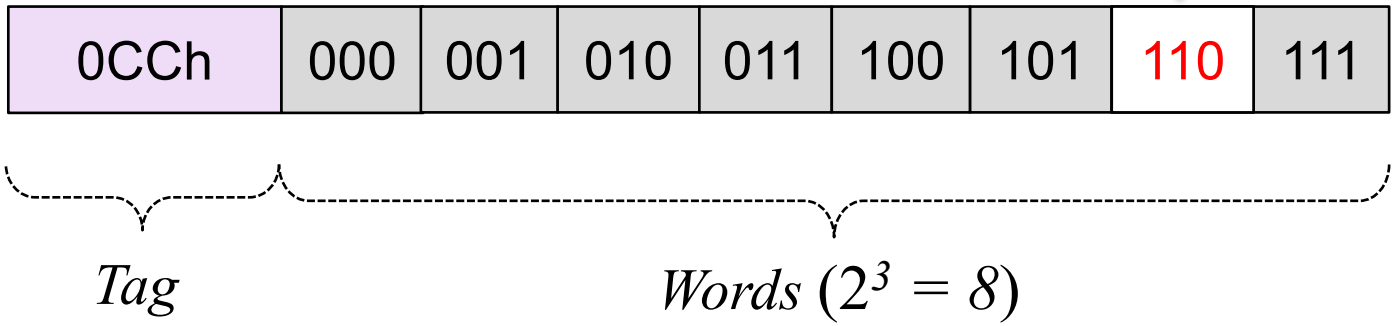
Supposed the program generates the memory address **666h**.

→ In 14-bit binary, this number is: 00 0110 0110 0110

→ 00 0110 0110 0110

It will find the data it is looking in cache memory for in tag 0CCh, *word* 110.

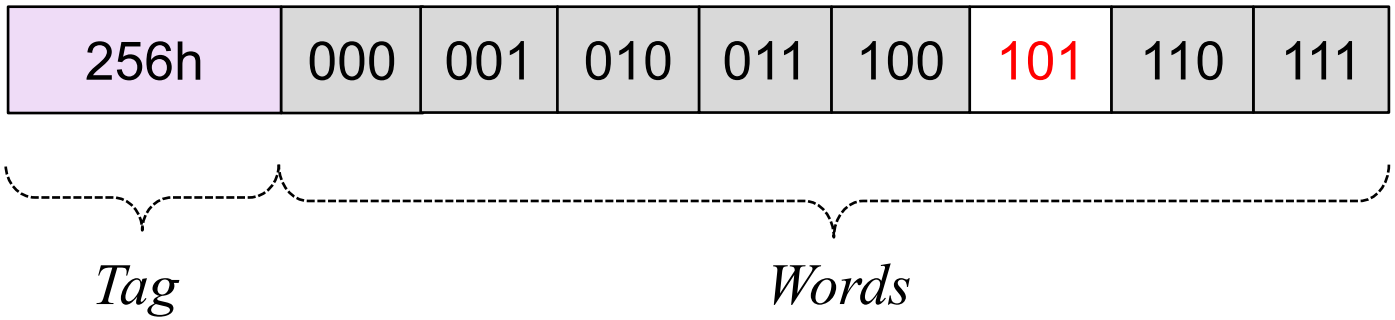
Cache Memory :



Example 18:

Consider a memory configuration with 2^{14} words, and blocks of 8 words. For data in cache as follows, what is the memory address (hexadecimal).

Cache Memory : (***Data***)



→ In binary, the number 256h is: 10 0101 0110

→ Thus, the memory address = 01 0010 1011 0101 (14-bit)

= **12B5**h

(d) Set Associative Mapping

6

- In this scheme, instead of mapping anywhere in the entire cache, a memory reference can map only to the subset of cache slots.
- Set Associative Mapping is a compromise that exhibits the strengths of both the direct mapping and fully associative mapping approaches while reducing their disadvantages.

Linda Null and Julia Lobur (2003). *The Essentials of Computer Organization and Architecture.* United States: Jones and Bartlett Publishers. p.246
William Stallings (2016). *Computer Organization and Architecture: Designing for Performance* (10th Edition). United States: Pearson Education Limited, p.140
*Patterson, D.A. and Hennessy, J.L. (2014). *Computer Organization and Design: The Hardware/Software Interface (*5th Edition). United States: Elsevier, p.403



- This scheme is similar to direct mapped cache, in that we use the address to map the block to a certain cache location.
- The important difference is that instead of mapping to a single cache block, an address maps to a set of several cache blocks.
- All sets in cache must be the same size.
- If there are 2 sets, it is called 2-way set associative cache mapping

Linda Null and Julia Lobur (2003). *The Essentials of Computer Organization and Architecture.* United States: Jones and Bartlett Publishers. p.246
William Stallings (2016). *Computer Organization and Architecture: Designing for Performance* (10th Edition). United States: Pearson Education Limited, p.140
*Patterson, D.A. and Hennessy, J.L. (2014). *Computer Organization and Design: The Hardware/Software Interface (5*th Edition). United States: Elsevier, p.403

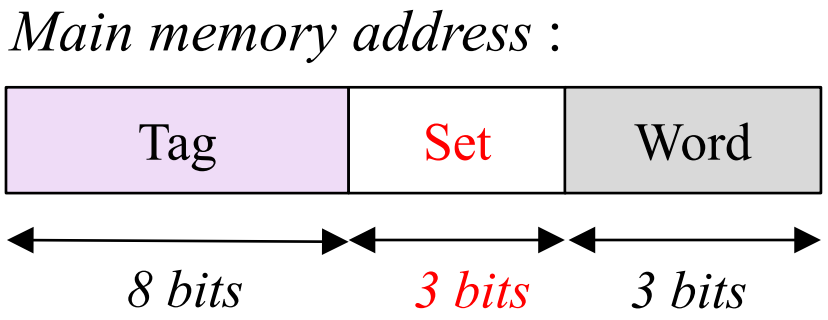
- In set-associative cache mapping, the main memory address is partitioned into three pieces:

Main memory address :



Linda Null and Julia Lobur (2003). *The Essentials of Computer Organization and Architecture.* United States: Jones and Bartlett Publishers. p.246
Patterson, D.A. and Hennessy, J.L. (2014). *Computer Organization and Design: The Hardware/Software Interface (5th* Edition). United States: Elsevier, p.403

Example 19: 2-way



- Suppose we have a main memory of 16K bytes (~ 16K = 2^{14})
- It is mapped to a 2-way set associative cache having 16 blocks where each block contains 8 words.

- Since this is a 2-way cache, each set consists of 2 blocks, and there are 8 sets (i.e., 16 blocks / 2 = 8 sets).
- Thus, we need 3 bits for the set, 3 bits for the word, giving 8 leftover bits for the tag.

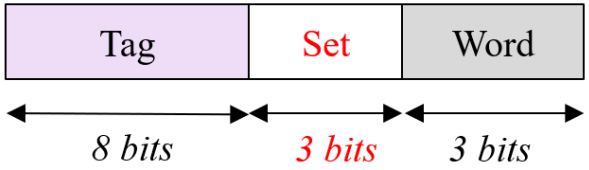
A set can be in 2 different blocks

Set	Tag	Block 0 of set	Valid	Tag	Block 1 of set	Valid
0	00000000	Words A, B, C, . . .	1	----------		0
1	11110101	Words L, M, N, . . .	1	----------		0
2	----------		0	10111011	P, Q, R, . . .	1
3	----------		0	11111100	T, U, V, . . .	1

Set 0 contains two blocks, one that is valid and holds the data A, B, C, . . . , and another block is not valid.

Given a tag, only search in the set :
If one of the 2 blocks matches ➔ it's a HIT.
Else ➔ it's a MISS
➔ go fetch from main memory

Main memory address :



Example 19 : 2-way

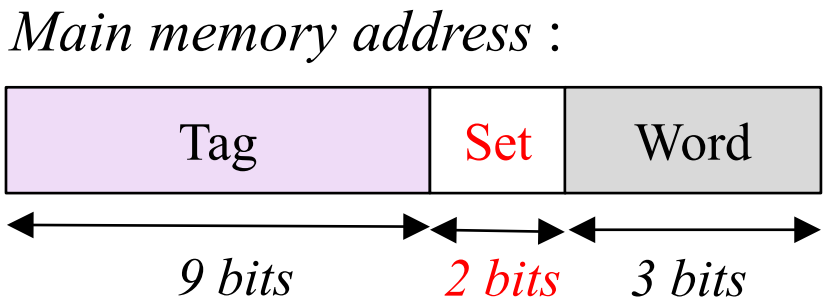
Cache memory (Hex.):

Set	*Tag*	*Word*	*Tag*	*Word*
7	FF	7	FF	6
7	FF	5	FF	4
7	FF	3	FF	2
7	FF	1	FF	0
6	FF	7	FF	6
6	FF	5	FF	4
6	FF	3	FF	2
6	FF	1	FF	0

(Hex.) *Memory Address:*

	Tag | *Set* | *Word*
3FFF	11111111111111
3FFE	11111111111110
3FFD	11111111111101
3FFC	11111111111100
3FFB	11111111111011
3FFA	11111111111010
3FF9	11111111111001
3FF8	11111111111000
3FF7	11111111110111
3FF6	11111111110110
3FF5	11111111110101
3FF4	11111111110100
3FF3	11111111110011
3FF2	11111111110010
3FF1	11111111110001
3FF0	11111111110000

Example 20: 4-way



- Suppose we have a main memory of 16K bytes (or 2^{14}).
- It is mapped to a 4-way set associative cache having 16 blocks where each block contains 8 words.
- Since this is a 4-way cache, each set consists of 4 blocks, and there are 4 sets (i.e., 16 blocks / 4 = 4 sets).
- Thus, we need 2 bits for the set, 3 bits for the word, giving 9 leftover bits for the tag.

(Hex.)	*Memory Address:*
3FFF	1111111111111
3FFE	1111111111110
3FFD	1111111111101
3FFC	1111111111100
3FFB	1111111111011
3FFA	1111111111010
3FF9	1111111111001
3FF8	1111111111000

(Hex.)	*Memory Address:*
3FF7	1111111110111
3FF6	1111111110110
3FF5	1111111110101
3FF4	1111111110100
3FF3	1111111110011
3FF2	1111111110010
3FF1	1111111110001
3FF0	1111111110000

Cache memory (Hex.):

Set	*Tag*	*Word*	*Tag*	*Word*	*Tag*	*Word*	*Tag*	*Word*
3	FF	7	FF	6	FF	5	FF	4
3	FF	3	FF	2	FF	1	FF	0
2	FF	7	FF	6	FF	5	FF	4
2	FF	3	FF	2	FF	1	FF	0

Figure: An eight-block cache configured as direct mapped, two-way set associative, four-way set associative, and fully associative.

One-way set associative (direct mapped)

Block	Tag	Data
0		000
1		001
2		010
3		011
4		100
5		101
6		110
7		111

Two-way set associative

Set	Tag	Data	Tag	Data
0		000		001
1		010		011
2		100		101
3		110		111

Four-way set associative

Set	Tag	Data	Tag	Data	Tag	Data	Tag	Data
0		000		001		010		011
1		100		101		110		111

Eight-way set associative (fully associative)

Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data
	000		001		010		011		100		101		110		111

6 Replacement Policy

- *Once the cache has been fully filled up, when a new block is brought into the cache, one of the existing blocks must be replaced.

How do we determine which block in cache should be replaced?



- The algorithm for determining replacement is called the *replacement policy*.

Linda Null and Julia Lobur (2003). *The Essentials of Computer Organization and Architecture.* United States: Jones and Bartlett Publishers. p.247

* William Stallings (2016). *Computer Organization and Architecture: Designing for Performance* (10th Edition). United States: Pearson Education Limited, p.145



- A replacement policy is invoked when it becomes necessary to evict or to remove a line/slot/block from cache.
- An *optimal* replacement policy would be able to look into the future to see *which blocks won't be needed for the longest period of time*.

* Linda Null and Julia Lobur (2003). *The Essentials of Computer Organization and Architecture.* United States: Jones and Bartlett Publishers. p.247
William Stallings (2016). *Computer Organization and Architecture: Designing for Performance* (10th Edition). United States: Pearson Education Limited, p.145

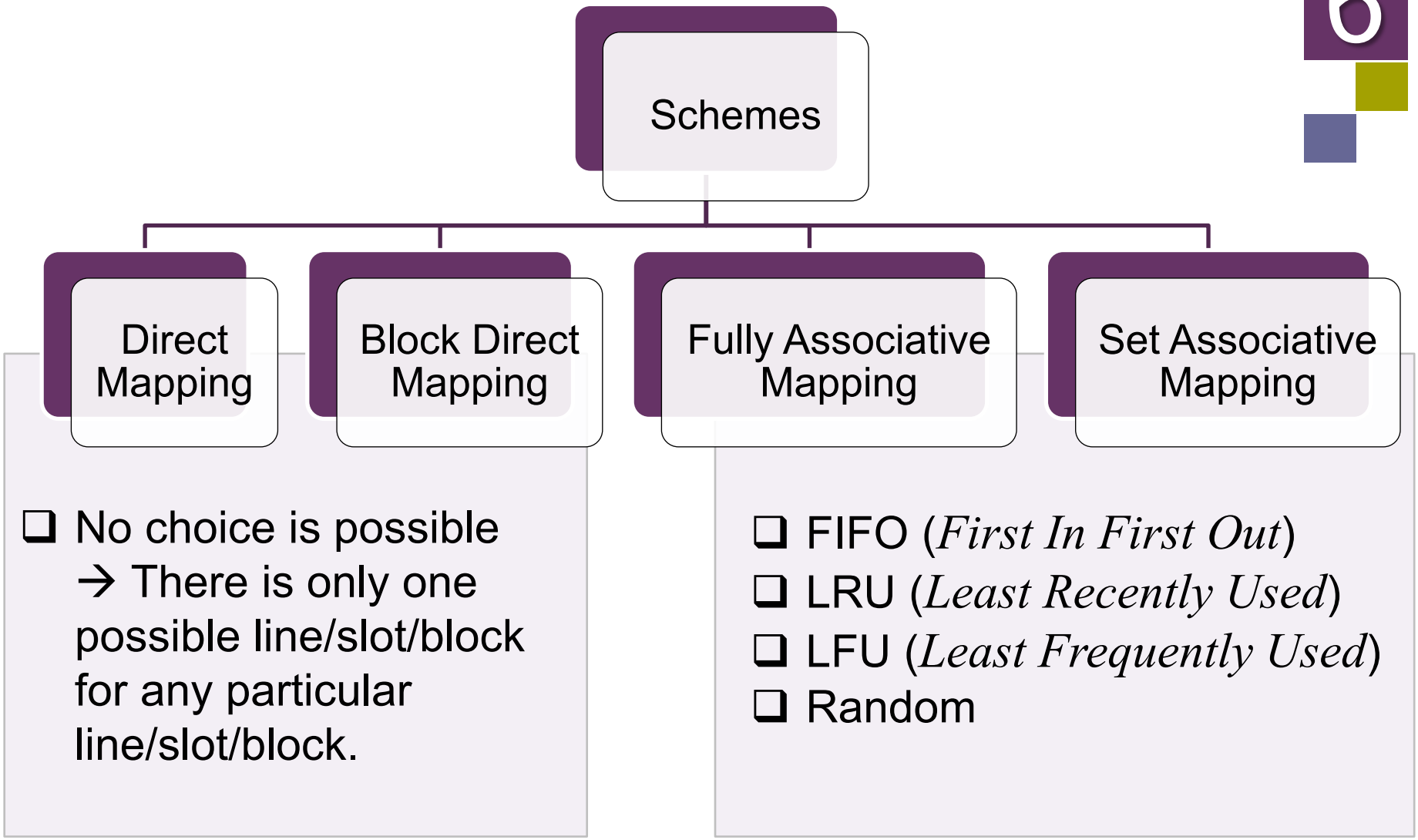


Figure: The replacement algorithms for cache memory.

Table: The replacement algorithms for cache memory.

Algorithms	Operational
FIFO *(First In First Out)*	Replace block that has been in cache longest.
LRU *(Least Recently Used)*	Keeps track of the last time that a block was assessed and evicts block that has been unused for the longest period of time.
LFU *(Least Frequently Used)*	Replace block which has had fewest hits.
Random	Picks a block at random and replaces it with a new block.

Linda Null and Julia Lobur (2003). *The Essentials of Computer Organization and Architecture.* United States: Jones and Bartlett Publishers. p.248
William Stallings (2016). *Computer Organization and Architecture: Designing for Performance* (10th Edition). United States: Pearson Education Limited, p.145

Cache Write Policies

- In addition to determining which victim slots / blocks to select for replacement, designers must also decide what to do with *dirty blocks* of cache (i.e., blocks that have been modified in cache)
- Dirty blocks must be written back to memory.
- A write policy determines how this will be done.

Linda Null and Julia Lobur (2003). *The Essentials of Computer Organization and Architecture.* United States: Jones and Bartlett Publishers. p.249
*William Stallings (2016). *Computer Organization and Architecture: Designing for Performance* (10th Edition). United States: Pearson Education Limited, p.145

- *There are two basic write policies:

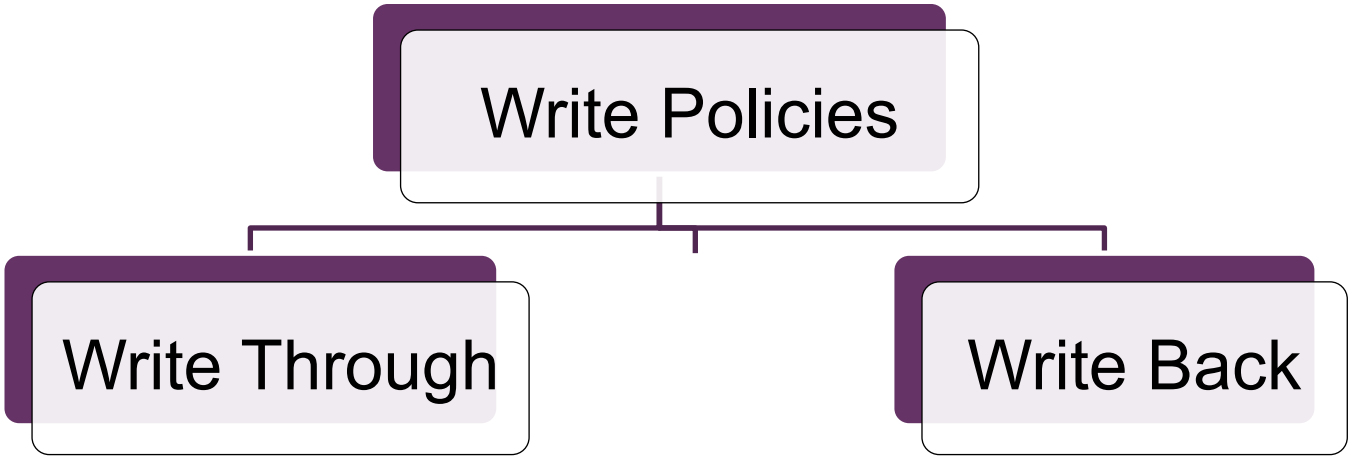


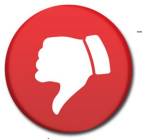
Figure: Cache write policy techniques

Linda Null and Julia Lobur (2003). *The Essentials of Computer Organization and Architecture.* United States: Jones and Bartlett Publishers. p.249
*William Stallings (2016). *Computer Organization and Architecture: Designing for Performance* (10th Edition). United States: Pearson Education Limited, p.145
Patterson, D.A. and Hennessy, J.L. (2014). *Computer Organization and Design: The Hardware/Software Interface (5th* Edition). United States: Elsevier, p.393

Cache Write Policies:

(a) Write Through

- A write-through policy updates both the cache and the main memory simultaneously on every write.



Every write to the cache requires a main memory access, essentially slows the system down to main memory speed.

- However, in real applications, the majority of accesses are reads so this slow-down is negligible.

Cache Write Policies:

(b) Write Back

- A write-back policy (also called *copyback*) only updates blocks in main memory when the cache block is selected as a victim and must be removed from cache.



- Normally faster than *write-through* because time is not wasted writing information to memory on each write to cache.
- Memory traffic is also reduced.



- Main memory & cache may not contain the same value at a given instant of time.
- Data in cache may be lost, if a process terminates (crashes) before the write to main memory is done.

6 Cache Performances

- To improve the performance of cache, we can increase the hit ratio by using :
  - a better mapping algorithm (up to roughly a **20%** increase),
  - better strategies for write operations (potentially a **15%** increase),
  - better replacement algorithms (up to a **10%** increase), and
  - better coding practices (up to a **30%** increase in hit ratio).
- Simply increasing the size of cache may improve the hit ratio by roughly ***1…4%*** but it is not guaranteed to do so.

Average Memory Access Time (AMAT)

- To access data for both hits and misses affects performance, designers sometime use AMAT to examine alternative cache designs.

$$\text{AMAT} = \textit{Time for a hit} + (\textit{Miss rate} \times \textit{Miss penalty})$$

- AMAT is the average time to access memory by considering both hits and misses and the frequency of different accesses.

Example 22:

Assume that 33% of the instructions in a program are data accesses. The cache hit ratio is 97% and the hit time is one cycle, but the miss penalty is 20 cycles.

$$\begin{aligned} \text{AMAT} &= \textit{Time for a hit} + (\textit{Miss rate} \times \textit{Miss penalty}) \\ &= 1\ \textit{cycle} + (3\% \times 20\ \textit{cycles}) \\ &= 1\ \textit{cycle} + (0.03 \times 20\ \textit{cycles}) \\ &= 1.6 \text{ cycles} \end{aligned}$$

If the cache was perfect and never missed, the AMAT would be one cycle. But even with just a 3% miss rate, the AMAT here increases 1.6 times!

Effective Access Time (EAT)

6

- The performance of a hierarchical memory is measured by its EAT, or the average time per access.
- EAT is a weighted average that uses the hit ratio and the relative access times of the successive levels of the hierarchy.

$$EAT = (H \times Access_C) + ((1 - H) \times Access_{MM})$$

H : Cache hit rate
$Access_C$: Access time for cache
$Access_{MM}$: Access time for main memory

This formula can be extended to apply to three- or even four-level memories

Linda Null and Julia Lobur (2003). *The Essentials of Computer Organization and Architecture.* United States: Jones and Bartlett Publishers. p.248
William Stallings (2016). *Computer Organization and Architecture: Designing for Performance* (10th Edition). United States: Pearson Education Limited, p.147

Example 23:

Suppose the cache access time is $10ns$, main memory access time is $200ns$, and the cache hit rate is 99%.

The average time for the processor to access an item in this two-level memory would then be:

$$\begin{aligned} EAT &= (H \times Access_C) + ((1-H) \times Access_{MM}) \\ &= (0.99 \times 10ns) + ((1-0.99) \times 200ns) \\ &= 9.9ns + (0.01)\,200ns \\ &= 9.9ns + 2ns = 11ns \end{aligned}$$

Module 6
Memory

6.1 Introduction
6.2 Main Memory
6.3 Cache Memory
6.4 Virtual Memory
6.6 Summary

- Overview
- A Real-World Example: Pentium

Overview

- We now know that caching allows a computer to access frequently used data from a smaller but faster cache memory. (Cache is found near the top of our memory hierarchy).
- Another important concept inherent in the hierarchy is *virtual memory*.

The purpose of virtual memory is to use the hard disk as an extension of RAM, thus increasing the available address space a process can use.



Paging

- The most common way to implement virtual memory is by using *paging*.
- Main memory is divided into fixed-size blocks and programs are divided into the same size blocks.
- Typically, chunks of the program are brought into memory as needed.
- It is not necessary to store contiguous chunks of the program in contiguous chunks of main memory.
- Every virtual address for a program that generated by CPU must be translated into a physical address.

Table: Some frequently used terms for virtual memory implemented through paging.

Terms	Description
Virtual address	The logical or program address generated by CPU.
Physical address	The real address in physical memory.
Mapping	Mechanism by which virtual addresses are translated into physical memory.
Page frames	The equal-size chunks in main memory.
Pages	The equal-size chunks in virtual memory.
Paging	The process of copying a virtual page from disk to a page frame in main memory.
Fragmentation	Memory that becomes unusable.
Page fault	An event when a requested page is not in main memory and must be copied into memory from disk.

- Virtual memory can be implemented with different techniques:

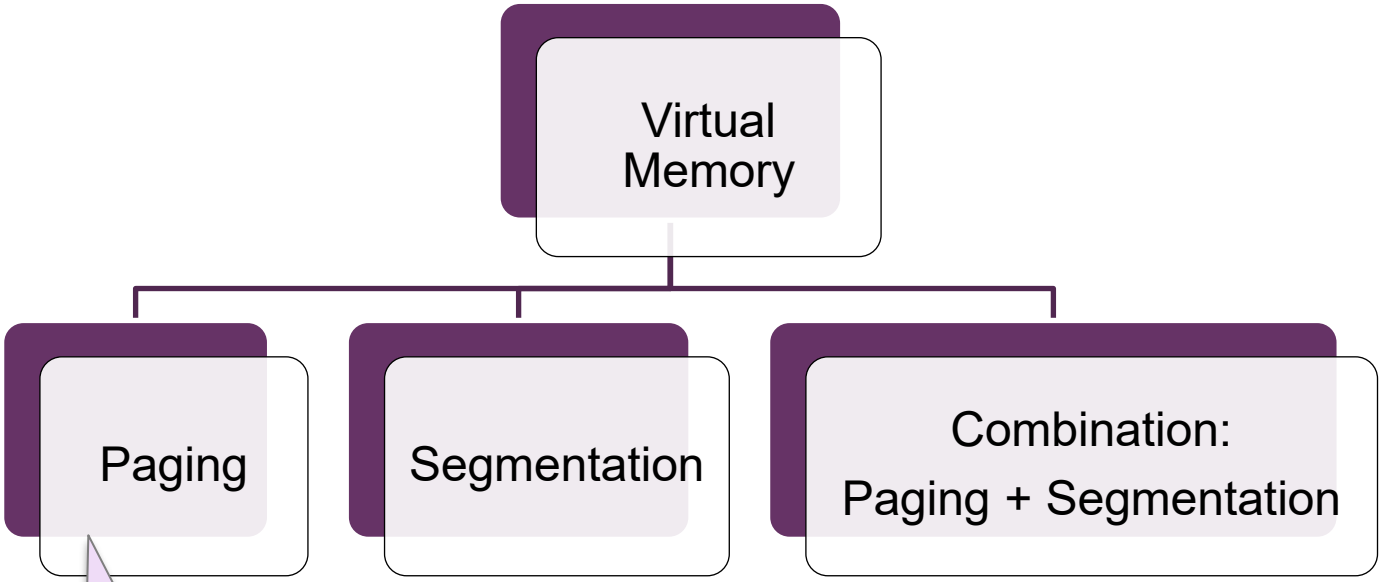


Figure: Virtual memory techniques

The most popular techniques !

A Real-World Example: Pentium

- The Pentium architecture exhibits fairly characteristic traits of modern memory management.
  - 32-bit virtual addresses and 32-bit physical addresses.
  - Uses either 4KB or 4MB page sizes, when using paging in different combinations of paging and segmentation.
  - Two caches, L1 and L2, both utilizing a 32-byte block size.
  - Both L1 caches (*I-cache* and *D-cache*) utilize an LRU bit for dealing with block replacement.
  - Each L1 cache has a TLB (*Translation Lookaside Buffer*).
  - The L2 cache can be from 512KB up to 1MB.

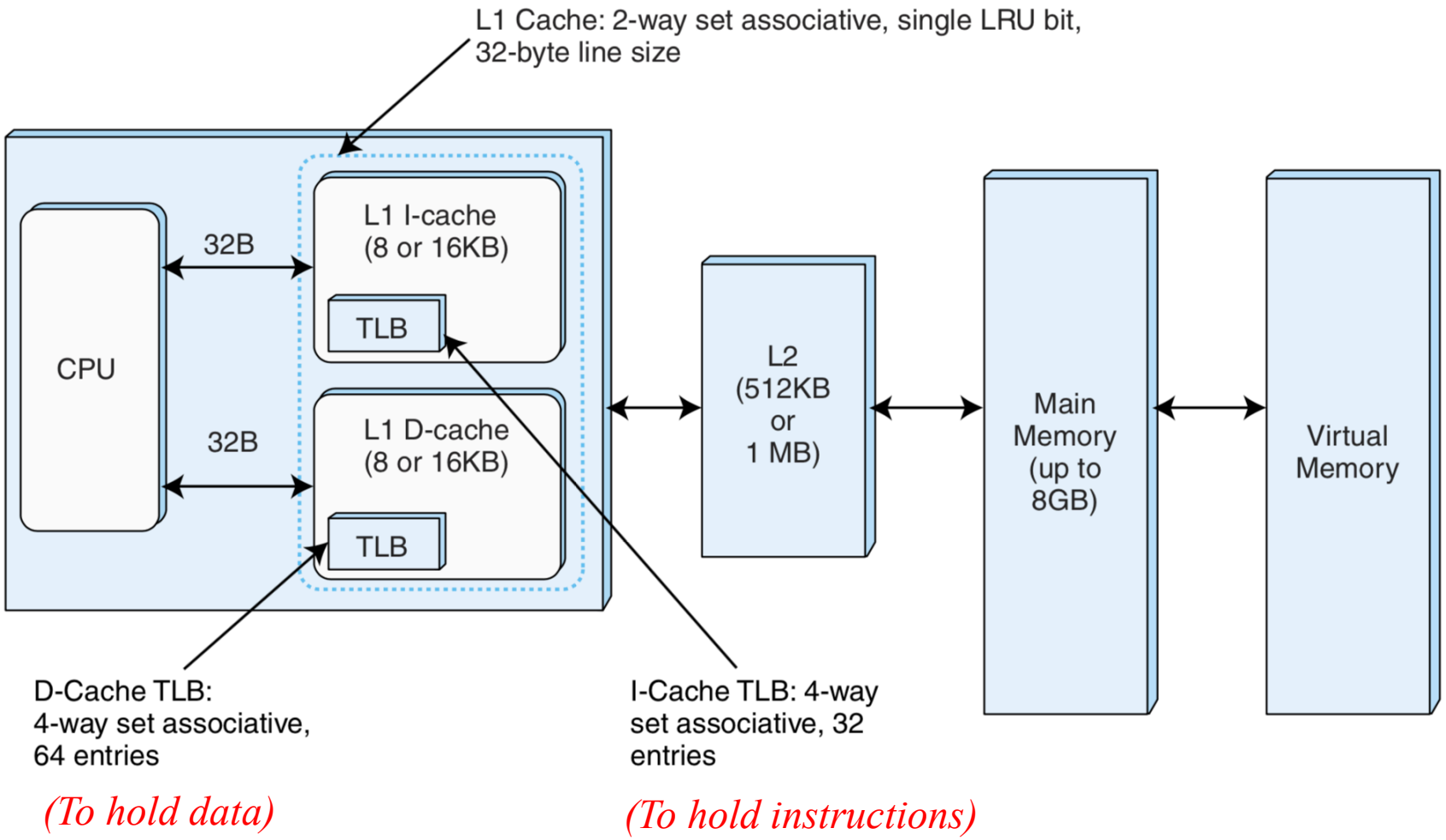


Figure: Pentium memory hierarchy

6.5 Summary

6

- Computer memory is organized in a hierarchy, with the smallest, fastest memory at the top and the largest, slowest memory at the bottom.
- Cache memory gives faster access to main memory, while virtual memory uses disk storage to give the illusion of having a large main memory.
- Cache maps blocks of main memory to blocks of cache memory.
- Virtual memory maps page frames to virtual pages.



- There are three general types of cache: Direct mapped, fully associative and set associative.
- With fully associative and set associative cache, as well as with virtual memory, replacement policies must be established.
- Replacement policies include LRU, FIFO, or LFU. These policies must also take into account what to do with dirty blocks.